รายงาน

โครงงานวิชา Object Oriented Data Structure

เรื่อง

Distributed Hilbert Hotel

จัดทำโดย

68010691 นาย ปัณณวิชญ์	กิติวงศ์ไพศาล
68010467 นาย ธนภัทร	แคล้วคลาด
68010544 นาย ธีรปรัชญ์	ตั้นพันธุ์
68010125 นาย คีตกานต์	แซ่ลิ่ม

เสนอ

ผศ.เกียรติณรงค์ ทองประเสริฐ

รายงานนี้เป็นส่วนหนึ่งของรายวิชา Object Oriented Data Structure

ภาคเรียนที่ 1 ปีการศึกษา 2569

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

บทนำ

โรงแรมอนันต์ของฮิลเบิร์ต (Hilbert's Hotel) เป็นการทดลองทางความคิดทางคณิตศาสตร์ที่ใช้อธิบายคุณสมบัติของเซตอนันต์ ในจินตนาการนี้โรงแรมมีห้องพักจำนวนไม่จำกัดและแขกเต็มทุกห้อง แต่ยังสามารถรับแขกเพิ่มได้เสมอ แม้จะเป็นแขกจำนวนอนันต์จากหลายช่องทางก็ตาม โดยให้แขกเดิมทุกคนย้ายห้องเพื่อเปิดห้องว่าง วิธีนี้ถูกต้องในเชิงคณิตศาสตร์ แต่ในระบบจริง การรับแขกหนึ่งคนแล้วต้องย้ายแขกทั้งหมดสิ้นเปลืองเกินไป โครงงานนี้จึงคำนวณเลขห้องจากรหัสของแขกโดยตรง เลขห้องจึงไม่ซ้ำกัน และการรับแขกใหม่ไม่ทำให้แขกเดิมต้องย้าย แนวคิดนี้สอดคล้องกับปัญหาจริงในระบบคอมพิวเตอร์ที่ต้องจัดเก็บและค้นหาข้อมูลจำนวนมากจากหลายแหล่ง โดยข้อมูลเพิ่มขึ้นได้ตลอดเวลาและระบบต้องตอบสนองได้ทันที

เมื่อจำนวนแขกมีมากเกินกว่าที่อาคารเดียวจะรองรับได้ จำเป็นต้องกระจายแขกไปยังอาคารหลายหลัง วิธีที่ง่ายที่สุดคือ hash mod N ซึ่งนำค่าแฮชของแขกมาหารเอาเศษด้วยจำนวนอาคาร แต่วิธีนี้มีข้อเสียสำคัญ เพราะเมื่อมีการเพิ่มหรือลบอาคาร ค่า N จะเปลี่ยน ทำให้แขกส่วนใหญ่ต้องย้ายไปอาคารอื่น ซึ่งสิ้นเปลืองทรัพยากรและไม่เหมาะกับระบบที่ต้องขยายหรือลดขนาดอยู่เสมอ

โครงงานนี้จึงนำแนวคิด Consistent Hashing มาใช้แก้ปัญหาดังกล่าว โดยแปลงรหัสแขก (c, s) และอาคารเป็นตำแหน่งบนวงแหวนแฮชด้วย SHA-256 แล้วให้แขกอยู่ในอาคารที่มีตำแหน่งถัดไปตามเข็มนาฬิกา นอกจากนี้ยังใช้ Virtual Node เพื่อให้แต่ละอาคารมีหลายจุดบนวงแหวน ช่วยให้การกระจายโหลดสมดุลมากขึ้น ผลคือเมื่อเพิ่มอาคารจะมีแขกย้ายเข้าอาคารใหม่เท่านั้น และเมื่อลบอาคาร จะมีเฉพาะแขกของอาคารนั้นที่ต้องย้าย ส่วนแขกที่เหลือยังคงอยู่ที่เดิมและใช้เลขห้องเดิม

โปรแกรมที่พัฒนาแบ่งเป็นสามส่วนตามหน้าที่ ได้แก่ ส่วนตรรกะหลักที่รับผิดชอบการเพิ่ม ลบ และค้นหาแขก การเพิ่มและลบอาคาร การคำนวณสถิติโหลด การรายงานการย้ายแขก และการส่งออกไฟล์ CSV ส่วนติดต่อผู้ใช้แบบเมนูข้อความใน terminal ที่รับคำสั่งและแสดงผลลัพธ์ และชุดการทดลองที่วัดเวลา หน่วยความจำ อัตราการย้ายแขก และความสมดุลของโหลด โดยเปรียบเทียบ Consistent Hashing กับ hash mod N เพื่อประเมินประสิทธิภาพของแนวทางที่เลือกใช้อย่างเป็นรูปธรรม

สารบัญ

	หน้า
บทนำ	ก
สารบัญ	ข
ภาพรวมของโครงงาน	1
การออกแบบเชิงวัตถุ	3
โครงสร้างข้อมูล	7
กติกาการคำนวณ	11
ส่วนติดต่อผู้ใช้และไฟล์	17
การวิเคราะห์ BigO	24
การทดลอง	29
การทดสอบความถูกต้อง	37
ข้อจำกัดและแนวทางพัฒนา	47
แหล่งอ้างอิง	49
ภาคผนวก	50

บทที่ 1
ภาพรวมของโครงงาน (Project Overview)

1.1 ที่มาของปัญหา

Hilbert's Hotel เป็นโรงแรมสมมติที่มีห้องพักจำนวนอนันต์ ในปัญหาดั้งเดิม เมื่อโรงแรมเต็มแล้วมีแขกใหม่เข้ามา แขกเดิมทุกคนต้องย้ายห้องเพื่อให้มีห้องว่าง วิธีนี้ถูกต้องในเชิงคณิตศาสตร์ แต่ระบบจริงใช้ไม่ได้ เพราะการรับแขกหนึ่งคนต้องย้ายแขกทั้งหมด

โครงงานนี้ขยายโรงแรมเป็นเครือโรงแรมที่มีหลายอาคาร แต่ละอาคารไม่จำกัดจำนวนห้อง โจทย์จึงไม่ใช่ "ย้ายแขกอย่างไรให้ครบ" แต่เป็น "ออกแบบอย่างไรให้แทบไม่ต้องย้ายเลย" วิธีที่ง่ายที่สุดคือเลือกอาคารด้วย hash(รหัสแขก) mod N แต่เมื่อเพิ่มหรือลบอาคาร ตัวหาร N จะเปลี่ยน ผลการหารเอาเศษจึงเปลี่ยนเกือบทุกคน แขกเกือบทั้งหมดต้องย้าย ซึ่งก็คือปัญหาเดิม

1.2 ภาพรวมของระบบ

โครงงานนี้พัฒนาโปรแกรมภาษา Python ที่คำนวณว่าแขกแต่ละคนพักอาคารใดและห้องใด แทนการจดบันทึกทีละคน แขกแต่ละคนระบุด้วยรหัส (c, s) โดย c คือช่องทางที่เดินทางมา และ s คือลำดับในช่องทางนั้น หมายเลขห้องคำนวณจากรหัสแขกโดยตรงด้วย Cantor pairing function จึงไม่ซ้ำกันและไม่ขึ้นกับแขกคนอื่น ส่วนอาคารเลือกด้วยวิธี Consistent Hashing คือวางทั้งอาคารและแขกบนวงแหวนแฮช แล้วให้แขกพักที่อาคารแรกที่พบเมื่อเดินตามเข็มนาฬิกา โดยแต่ละอาคารมีหลายจุดบนวงแหวน (virtual node) เพื่อให้กระจายโหลดได้สมดุลขึ้น ผลที่ได้คือ เมื่อมีแขกใหม่เข้ามา แขกเดิมไม่ต้องย้ายเลย และเมื่อเพิ่มหรือลบอาคาร จะมีแขกย้ายเพียงส่วนน้อยเมื่อเทียบกับวิธี hash mod N โดยแขกที่ย้ายจะเปลี่ยนเฉพาะอาคาร หมายเลขห้องยังเป็นเลขเดิม และโปรแกรมรายงานได้ว่าใครย้ายจากอาคารใดไปอาคารใด โปรแกรมเพิ่ม ลบ และค้นหาแขกได้ทั้งจากรหัสแขกและจากที่อยู่ห้อง รายงานความสมดุลของโหลดของแต่ละอาคาร และส่งออกข้อมูลเป็นไฟล์ CSV นอกจากนี้ยังมีชุดการทดลองที่วัดเวลา หน่วยความจำ อัตราการย้าย และความสมดุลของโหลด เพื่อเปรียบเทียบ Consistent Hashing กับวิธี hash mod N

1.3 ขอบเขตของโครงงาน

- โปรแกรมทั้งหมดรันบนเครื่องเดียว อาคารแต่ละหลังแทนด้วยรหัสอาคาร (node_id) ภายในโปรแกรมเดียว ไม่มีระบบเครือข่ายหรือฐานข้อมูล
- ไม่จำลองการล่มของอาคารหรือการกู้คืนข้อมูล
- คำว่า "อนันต์" หมายถึงห้องพักไม่มีความจุสูงสุดที่กำหนดไว้ล่วงหน้า ข้อมูลที่ประมวลผลได้ในแต่ละครั้งมีจำนวนจำกัดตามทรัพยากรของเครื่อง
- ส่วนติดต่อผู้ใช้เป็นเมนูข้อความใน terminal

1.4 โครงสร้างของโปรแกรม

โปรแกรมแบ่งเป็นส่วนต่าง ๆ ตามหน้าที่ ดังตารางที่ 1.1

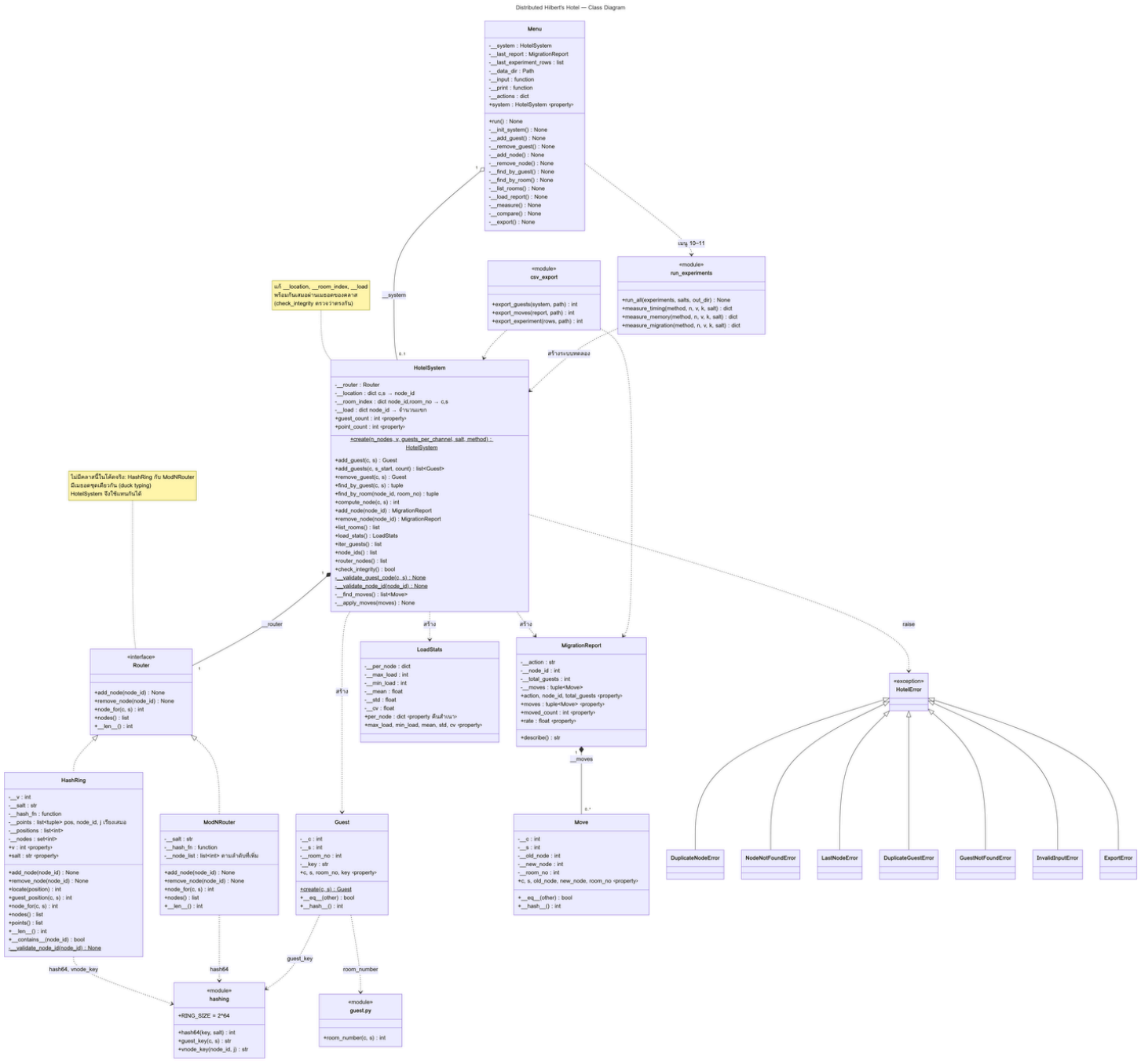
โฟลเดอร์ / ไฟล์	**หน้าที่**
main.py	จุดเริ่มโปรแกรม เรียกเมนูข้อความ
hotel/	ส่วนตรรกะหลัก: ฟังก์ชันแฮช วงแหวน ข้อมูลแขกและเลขห้อง การเพิ่ม/ลบอาคาร และการย้าย สถิติ และการส่งออก CSV ไม่มีการรับค่าหรือแสดงผล
cli/	ส่วนติดต่อผู้ใช้แบบเมนูข้อความ รับคำสั่ง เรียกส่วนตรรกะหลัก และแสดงผล
experiments/	ชุดการทดลอง A, B, C และการวาดกราฟ ผลลัพธ์เก็บใน experiments/results/
tests/	สคริปต์ทดสอบอัตโนมัติ 147 กรณี (unittest)
data/	ไฟล์ CSV ตัวอย่าง: ข้อมูลแขกและรายการย้าย

ตารางที่ 1.1 โครงสร้างโฟลเดอร์ของโปรแกรม

บทที่ 2
การออกแบบเชิงวัตถุ

โปรแกรมแบ่งออกเป็น 3 ส่วนตามหน้าที่ ส่วนแรกคือตรรกะหลักในโฟลเดอร์ hotel/ ซึ่งไม่มีการรับค่าหรือแสดงผลเลย ส่วนที่สองคือเมนูข้อความในโฟลเดอร์ cli/ ทำหน้าที่รับคำสั่งจากผู้ใช้ เรียกใช้ตรรกะหลัก แล้วแสดงผล ส่วนที่สามคือการทดลองในโฟลเดอร์ experiments/ ซึ่งเรียกใช้ตรรกะหลักชุดเดียวกันโดยไม่ผ่านเมนู

2.1 แผนภาพคลาส



รูปที่ 2.1 แผนภาพคลาสของระบบ

Menu ใช้งาน HotelSystem เพียงคลาสเดียว ส่วน HotelSystem เก็บตัวเลือกอาคาร (router) ไว้หนึ่งตัว ซึ่งเป็น HashRing หรือ ModNRouter ก็ได้ ทั้งสองคลาสคำนวณตำแหน่งด้วยฟังก์ชัน hash64 HotelSystem สร้างวัตถุ Guest และส่งผลลัพธ์กลับเป็น MigrationReport (ภายในมีรายการ Move) และ LoadStats เมื่อทำตามคำสั่งไม่ได้ จะแจ้งข้อผิดพลาดด้วยคลาสในกลุ่ม HotelError

2.2 หน้าที่ของแต่ละคลาส

คลาส	ไฟล์	หน้าที่	ข้อมูลที่เก็บ	เมธอดหลัก
HashRing	hotel/hash_ring.py	วงแหวน Consistent Hashing: วาง virtual node และหาอาคารของแขก	__points, __positions, __nodes, __v, __salt	add_node, remove_node, locate, node_for, nodes
ModNRouter	hotel/mod_n.py	เลือกอาคารด้วย H(guest) mod N ใช้เป็นวิธีเทียบในการทดลอง	__node_list (รหัสอาคารตามลำดับที่เพิ่ม), __salt	add_node, remove_node, node_for, nodes
Guest	hotel/guest.py	ข้อมูลแขกหนึ่งคน สร้างแล้วแก้ค่าไม่ได้	__c, __s, __room_no, __key	create (คำนวณเลขห้องจากรหัสแขก)
HotelSystem	hotel/hotel_system.py	ศูนย์กลางของระบบ: เพิ่ม/ลบแขก เพิ่ม/ลบอาคาร ย้ายแขก ค้นหา และสถิติ	__router, __location, __room_index, __load	add_guest, add_guests, remove_guest, add_node, remove_node, find_by_guest, find_by_room, list_rooms, load_stats, check_integrity
Move	hotel/models.py	แขกที่ย้ายหนึ่งรายการ	c, s, old_node, new_node, room_no	—
MigrationReport	hotel/models.py	ผลของคำสั่งเพิ่มหรือลบอาคาร	action, node_id, total_guests, moves	moved_count, rate, describe
LoadStats	hotel/models.py	สถิติความสมดุลของโหลด	per_node, max_load, min_load, mean, std, cv	—
HotelError และคลาสลูก 7 คลาส	hotel/models.py	ข้อผิดพลาดตามกติกา เช่น DuplicateNodeError, LastNodeError, GuestNotFoundError	—	—

Menu	cli/menu.py	เมนูข้อความ: รับค่า เรียก HotelSystem และแสดงผล	__system, __last_report, __actions	run และเมนูย่อย 12 ข้อ

ตารางที่ 2.1 หน้าที่ของคลาส

ส่วนที่ไม่มีข้อมูลต้องเก็บ เขียนเป็นฟังก์ชันระดับโมดูลแทนคลาส ได้แก่ hash64, guest_key, vnode_key ใน hashing.py, room_number ใน guest.py และ export_guests, export_moves, export_experiment ใน csv_export.py เพราะฟังก์ชันเหล่านี้รับค่าเข้าแล้วคืนผลลัพธ์เท่านั้น

2.3 หลักการออกแบบ

1) แยกตรรกะออกจากส่วนติดต่อผู้ใช้ ไฟล์ใน hotel/ ไม่มีคำสั่ง print หรือ input เลย เมื่อทำตามคำสั่งไม่ได้ จะ raise ข้อผิดพลาดในกลุ่ม HotelError ให้ Menu จับไปแสดงเป็นข้อความ โปรแกรมจึงไม่หยุดทำงาน สคริปต์ทดสอบและการทดลองจึงเรียก HotelSystem ได้โดยตรงโดยไม่ต้องผ่านเมนู นอกจากนี้ Menu ยังรับฟังก์ชันรับค่าและแสดงผลเป็นพารามิเตอร์ (input_fn, print_fn) เทสต์จึงจำลองการพิมพ์ของผู้ใช้ได้

2) ใช้ตัวเลือกอาคารแทนกันได้โดยไม่ใช้ inheritance HashRing กับ ModNRouter มีเมธอดชุดเดียวกัน คือ add_node, remove_node, node_for และ nodes ดังนั้น HotelSystem จึงรับตัวใดก็ได้ การทดลองเทียบสองวิธีจึงใช้โค้ด HotelSystem ชุดเดียวกัน ต่างกันเฉพาะวิธีเลือกอาคาร การเปรียบเทียบจึงเป็นธรรม ส่วน inheritance ใช้เฉพาะกับข้อผิดพลาด ซึ่งทุกคลาสสืบทอดจาก HotelError ทำให้ Menu จับข้อผิดพลาดทุกชนิดได้ในจุดเดียว

3) Encapsulation แอตทริบิวต์ภายในทุกคลาสขึ้นต้นด้วย __ โค้ดนอกคลาสจึงอ่านได้ผ่าน property เท่านั้น และแก้ข้อมูลได้ผ่านเมธอดของคลาสเท่านั้น คลาสที่เป็นผลลัพธ์ (Guest, Move, MigrationReport, LoadStats) สร้างแล้วแก้ค่าไม่ได้ เช่น MigrationReport.moves คืนค่าเป็น tuple และ LoadStats.per_node คืนค่าเป็น สำเนา โค้ดภายนอกจึงทำให้ข้อมูลกับดัชนีไม่ตรงกันไม่ได้

2.4 ตัวอย่างการปรับข้อมูลหลายส่วนให้สอดคล้องกัน

HotelSystem เก็บข้อมูลแขกไว้ 3 ชุด และทั้ง 3 ชุดต้องตรงกันตลอดเวลา

- __location : (c, s) → node_id ใช้ค้นหาจากรหัสแขก
- __room_index : (node_id, room_no) → (c, s) ใช้ค้นหาจากที่อยู่ห้อง
- __load : node_id → จำนวนแขก ใช้รายงานความสมดุลของโหลด

ตัวอย่าง เริ่มระบบด้วย 3 อาคาร V = 32 แขก 3 ช่องทาง ช่องทางละ 5 คน (K = 15, ไม่ใช้ salt) แล้วเพิ่มอาคาร 4 มีแขกย้าย 5 คน (อัตราย้าย 0.333) หนึ่งในนั้นคือแขก (1, 5) ห้อง 15 ซึ่งย้ายจากอาคาร 3 ไปอาคาร 4

ข้อมูล	ก่อน	หลัง
__location[(1, 5)]	3	4
__room_index	(3, 15) → (1, 5)	(4, 15) → (1, 5) และลบ (3, 15) ออกแล้ว
__load ทั้งระบบ	{1: 5, 2: 3, 3: 7}	{1: 4, 2: 3, 3: 3, 4: 5}

ตารางที่ 2.2 ข้อมูลของแขก (1, 5) ก่อนและหลังเพิ่มอาคาร 4

หมายเลขห้องยังเป็น 15 เหมือนเดิม เปลี่ยนเฉพาะรหัสอาคาร (หัวข้อ 4.4) เมธอด __apply_moves ปรับข้อมูลทั้ง 3 ชุดในคราวเดียว

```
def __apply_moves(self, moves: list) -> None:

    for m in moves:
        del self.__room_index[(m.old_node, m.room_no)]
    for m in moves:
        self.__location[(m.c, m.s)] = m.new_node
        self.__room_index[(m.new_node, m.room_no)] = (m.c, m.s)
        self.__load[m.old_node] -= 1
        self.__load[m.new_node] += 1
```

รูปที่ 2.2 ฟังก์ชันการปรับข้อมูลให้สอดคล้องกัน

ถ้าลืมลบ (3, 15) ออกจาก __room_index การค้นด้วยที่อยู่ห้อง (3, 15) จะยังตอบว่าเป็นแขก (1, 5) ทั้งที่แขกย้ายไปอาคาร 4 แล้ว แขกหนึ่งคนจึงมีสองห้อง ถ้าลืมปรับ __load รายงานโหลดก็จะผิด ระบบจึงมีเมธอด check_integrity ไว้ตรวจว่าข้อมูลทั้ง 3 ชุดมีจำนวนเท่ากัน ดัชนีของแขกทุกคนชี้กลับมาที่แขกคนเดิม และจำนวนแขกต่ออาคารตรงกับที่นับได้จริง ส่วนเทสต์ test_search_still_correct_after_migration เพิ่มอาคารแล้วตรวจแขกทุกคนว่า find_by_guest, find_by_room และการคำนวณอาคารจากวงแหวนให้คำตอบตรงกัน ผลจากการรันตัวอย่างข้างต้น: ค้นแขก (1, 5) ได้ (4, 15), ค้นห้อง (4, 15) ได้แขก (1, 5), ค้นห้อง (3, 15) ไม่พบ และ check_integrity คืนค่า True

บทที่3
โครงสร้างข้อมูล

โครงงานนี้เลือกใช้โครงสร้างข้อมูลมาตรฐานของภาษา Python ได้แก่ list, dict และโมดูล bisect เป็นหลัก เนื่องจากข้อกำหนดของโครงงานไม่อนุญาตให้ใช้ไลบรารี Consistent Hashing สำเร็จรูป ตรรกะของวงแหวน การเลือกอาคาร และการย้ายแขกจึงต้องเขียนขึ้นเองทั้งหมด โครงสร้างข้อมูลของระบบแบ่งเป็นสามส่วน ได้แก่ วงแหวนแฮช ข้อมูลแขก และตัวกำหนดอาคารแบบ hash mod N สำหรับเปรียบเทียบ ดังรายละเอียดต่อไปนี้

3.1 วงแหวนแฮช (HashRing)

3.1.1 โครงสร้าง คลาส HashRing เก็บจุดบนวงแหวนด้วย list สองตัวที่มีลำดับตรงกัน

ตัวแปร	ชนิดข้อมูล	ข้อมูลที่เก็บ
__points	list ของ tuple (pos, node_id, j)	จุดของ virtual node ทุกจุด เรียงตาม (pos, node_id, j) ตลอดเวลา
__positions	list ของจำนวนเต็ม	ค่า pos ของแต่ละจุดใน __points ตามลำดับเดียวกัน ใช้เป็นอาร์กิวเมนต์ของ bisect_left

ตารางที่ 3.1 โครงสร้างคลาส HashRing

3.1.2 การค้นหาอาคารของแขก การหาอาคารของแขกรหัส (c, s) ดำเนินการดังนี้ ขั้นแรกคำนวณตำแหน่งของแขก p = hash64("guest:<c>:<s>", salt) จากนั้นใช้ bisect_left บน __positions เพื่อหาจุดแรกที่มีตำแหน่งมากกว่าหรือเท่ากับ p หากค้นเลยจุดสุดท้ายของวงแหวนจะวนกลับไปใช้จุดแรก และอาคารเจ้าของจุดที่ได้คืออาคารของแขกคนนั้น

3.1.3 เหตุผลในการเลือกโครงสร้าง เลือกใช้ list ที่เรียงลำดับร่วมกับ bisect เพราะรูปแบบการใช้งานของวงแหวนคือค้นหาบ่อยมาก แต่เปลี่ยนแปลงนาน ๆ ครั้ง ทุกครั้งที่เพิ่มแขก คำนวณอาคาร หรือตรวจการย้าย ต้องหาจุดถัดไปบนวงแหวน ซึ่ง bisect_left ทำได้ใน O(log M) ส่วนการเพิ่มหรือลบจุดเกิดเฉพาะตอนเพิ่มหรือลบอาคาร มีต้นทุน O(M) ถึง O(M log M) ต่อครั้ง แต่เกิดไม่บ่อย และ M = N × V มีเพียงหลักร้อย ต้นไม้ค้นหาแบบสมดุลเพิ่มหรือลบจุดได้เร็วกว่า แต่ค้นหาได้ O(log M) เท่ากัน และไม่มีในไลบรารีมาตรฐานของ Python ต้องเขียนเองทั้งหมด จึงไม่คุ้มสำหรับรูปแบบการใช้งานนี้

การเก็บแต่ละจุดเป็น (pos, node_id, j) แทนที่จะเก็บเฉพาะ pos มีเหตุผลเพื่อรองรับกรณีที่ค่าแฮชของ virtual node สองจุดชนกัน ระบบจะเก็บจุดที่ชนกันไว้ครบทุกจุด และเรียงลำดับได้แน่นอนด้วย node_id แล้วจึง j ผลการค้นหาจึงไม่ขึ้นกับลำดับที่เพิ่มอาคาร ซึ่งตรวจสอบแล้วด้วยเทสต์ test_add_order_does_not_matter และชุดเทสต์ TestCollisions

การแยกเก็บ __positions เป็น list ของตำแหน่งล้วน เรียงลำดับเดียวกับ __points เพราะตำแหน่งของแขกเป็นจำนวนเต็ม bisect_left จึงเทียบกับ list ของจำนวนเต็มได้โดยตรง ไม่ต้องสร้าง tuple ขึ้นมาเทียบทุกครั้ง และเทียบจำนวนเต็มได้เร็วกว่าเทียบ tuple ดัชนีที่ได้ใช้เปิดจุดใน __points ได้ทันที เมื่อหลายจุดอยู่ตำแหน่งเดียวกัน bisect_left คืนจุดแรกในกลุ่มนั้น ซึ่งเป็นจุดที่ node_id และ j น้อยที่สุดตามการเรียงของ __points ตรงกับกติกาจุดชนกัน

การเพิ่มหรือลบอาคารแตะต้องเฉพาะ V จุดของอาคารนั้น จุดของอาคารอื่นไม่เปลี่ยนตำแหน่ง แขกที่ไม่เกี่ยวข้องจึงไม่ต้องย้าย ซึ่งเป็นคุณสมบัติหลักของ Consistent Hashing และตรวจสอบด้วยเทสต์ test_remaining_points_unchanged_after_add_and_remove

3.2 ข้อมูลแขก (HotelSystem)

3.2.1 โครงสร้าง

คลาส HotelSystem เก็บสถานะของระบบด้วย dict สามตัว ดังตารางที่3.2 ตัวแปรทั้งสามเป็นตัวแปรภายในแบบ __name-mangled เพื่อป้องกันไม่ให้โค้ดภายนอกแก้ไขได้โดยตรง

ตัวแปร	key → value	หน้าที่
__location	(c, s) → node_id	ค้นหาด้วยรหัสแขก และตรวจรหัสซ้ำ
__room_index	(node_id, room_no) → (c, s)	ค้นหาด้วยที่อยู่ห้อง (ดัชนีย้อนกลับ)
__load	node_id → จำนวนแขก	เก็บโหลดของแต่ละอาคาร รวมอาคารที่มี 0 คน

ตารางที่ 3.2 ตัวแปรหลักของ HotelSystem

3.2.2 เหตุผลในการเลือกโครงสร้าง ระบบต้องค้นหาได้สองทาง คือรู้รหัสแขกแล้วหาห้อง และรู้ห้องแล้วหาแขก หากใช้ dict ตัวเดียว การค้นหาอีกทางหนึ่งต้องไล่ตรวจแขกทั้งหมดซึ่งใช้เวลา O(K) ผู้พัฒนาจึงเพิ่มดัชนีย้อนกลับ __room_index ทำให้ทั้งสองทางใช้เวลา O(1) โดยแลกกับหน่วยความจำที่เพิ่มขึ้นและภาระในการอัปเดตข้อมูลให้ตรงกัน

__location เก็บเฉพาะ node_id โดยไม่เก็บเลขห้อง เนื่องจากเลขห้องคำนวณได้จากรหัสแขกเสมอด้วยฟังก์ชัน Cantor pairing

$$\text{room_no} = (a + b)(a + b + 1)// 2 + b + 1 \text{ เมื่อ } a = c - 1 \text{ และ } b = s - 1$$

ฟังก์ชันนี้ใช้เลขจำนวนเต็มล้วนและเป็นฟังก์ชันหนึ่งต่อหนึ่ง เลขห้องของแขกต่างคนจึงไม่ซ้ำกัน และเลขห้องไม่เปลี่ยนเมื่อแขกย้ายอาคาร ระบบจึงไม่ต้องใช้วิธี "เลขห้องล่าสุด + 1" ซึ่งอาจทำให้เลขห้องของแขกเดิมเปลี่ยนเมื่อมีการเพิ่มแขกใหม่

__load แยกเก็บต่างหากเพื่อไม่ต้องนับแขกใหม่ทุกครั้งที่ขอรายงานความสมดุลของโหลด และเพื่อให้ทุกอาคารมีรายการอยู่แม้ไม่มีแขก ซึ่งจำเป็นต่อการคำนวณค่าต่ำสุด ค่าเฉลี่ย ส่วนเบี่ยงเบนมาตรฐาน และสัมประสิทธิ์การแปรผัน (CV) ให้ถูกต้อง

3.2.3 ความสอดคล้องของข้อมูล ทุกการเพิ่ม ลบ หรือย้ายแขก ต้องอัปเดต dict ทั้งสามตัวพร้อมกัน เงื่อนไขที่ระบบต้องรักษาคือ len(__location) = len(__room_index) = Σ __load.values() ซึ่งตรวจสอบด้วยเมธอด check_integrity() และเทสต์เรียกใช้เมธอดนี้ทั้งหลังการเพิ่ม/ลบอาคาร และหลังการถูกปฏิเสธคำสั่ง เพื่อยืนยันว่าข้อมูลเดิมไม่เปลี่ยนแปลง

นอกจากนี้ทุกเมธอดตรวจสอบความถูกต้องของคำสั่งทั้งหมดก่อนแก้ไขข้อมูล หากพบข้อผิดพลาดจะ raise ข้อผิดพลาดในตระกูล HotelError ทำให้ข้อมูลเดิมไม่ถูกแก้ไข เช่น การเพิ่มแขกเป็นกลุ่มที่มีรหัสซ้ำเพียงบางรายการจะถูกปฏิเสธทั้งกลุ่ม

3.2.4 การย้ายแขกเมื่อเพิ่มหรือลบอาคาร เมื่อมีการเพิ่มหรือลบอาคาร ระบบหาแขกที่ต้องย้ายโดยวนตรวจแขกทุกคนว่าอาคารที่คำนวณได้จากวงแหวนใหม่ต่างจากอาคารที่เก็บไว้หรือไม่ ถ้าต่างจึงนับเป็นการย้าย และบันทึกเป็นออบเจกต์ Move(c, s, old_node, new_node, room_no) รวบรวมไว้ใน MigrationReport โดยจำนวนผู้ย้ายเท่ากับ len(moves) เสมอ

3.2.5 ความซับซ้อนของเวลา

กำหนดให้ K คือจำนวนแขก N คือจำนวนอาคาร และ M = N × V

การทำงาน	เวลา
ค้นหาด้วยรหัสแขก / ค้นหาด้วยที่อยู่ห้อง	O(1) เฉลี่ย
หาอาคารของแขก (node_for)	O(log M)
เพิ่ม / นำแขกออก (รายคน)	O(log M) / O(1) เฉลี่ย
เพิ่มแขกเป็นกลุ่ม count คน	O(count · log M)
เพิ่ม / ลบอาคาร (ตรวจแขกทุกคน)	O(M log M + K log M + B log B) กรณีเลวร้ายสุด (หัวข้อ 6.8)
รายงานโหลด	O(N)
รายการห้องที่มีผู้พัก (เรียงลำดับ)	O(K log K)
หน่วยความจำรวม	O(K + M)

ตารางที่ 3.3 ความซับซ้อนของการทำงานหลัก

3.3 ตัวกำหนดอาคารแบบ hash mod N (ModNRouter)

3.3.1 โครงสร้างและหน้าที่ ModNRouter เป็นตัวกำหนดอาคารสำหรับเปรียบเทียบ (baseline) โดยเก็บ list __node_list ของรหัสอาคารตามลำดับที่เพิ่ม และเลือกอาคารของแขกจากสูตร

node = __node_list[hash64(“guest:⟨c⟩:⟨s⟩”, salt) mod N]

การเพิ่มอาคารคือการต่อท้าย list ส่วนการลบอาคารคือการนำรหัสนั้นออกโดยรักษาลำดับของที่เหลือ เมื่อ N เปลี่ยนและดัชนีของอาคารที่อยู่ท้ายจุดที่ลบเลื่อนไป แขกจำนวนมากจะได้อาคารใหม่ ซึ่งเป็นจุดอ่อนของวิธีนี้ที่โครงงานต้องการแสดงเทียบกับ Consistent Hashing

3.3.2 การออกแบบเพื่อการเปรียบเทียบที่เป็นธรรม

- interface เดียวกับ HashRing: มีเมธอด node_for, add_node, remove_node และ nodes ตามหลัก duck typing ทำให้ HotelSystem สลับใช้ได้โดยไม่ต้องแก้โค้ด ผ่านพารามิเตอร์ method="ring" หรือ "modn" ของ HotelSystem.create
- ใช้แฮชและ salt เดียวกัน: ทั้งสองวิธีวางแขกชุดเดียวกันด้วยค่าแฮชเดียวกัน ความแตกต่างของจำนวนผู้ย้ายจึงเกิดจากวิธีเลือกอาคารเท่านั้น (ตรวจโดยเทสต์ test_same_salt_same_hash_for_both_methods)
- นับการย้ายด้วยรหัสอาคารจริง: หลังการลบอาคาร ดัชนีของอาคารที่เหลือเลื่อน หากแขกยังอยู่อาคารเดิมต้องไม่นับว่าย้าย (ตรวจโดยเทสต์ test_modn_compares_by_real_node_id_not_index)

3.3.3 ความซับซ้อน การเลือกอาคาร node_for ใช้เวลา O(1) การเพิ่มและลบอาคารระดับตัวกำหนดใช้เวลา O(N) และใช้หน่วยความจำ O(N) ส่วนการเพิ่ม/ลบอาคารระดับระบบยังต้องตรวจแขกทุกคนเช่นเดียวกับวงแหวน คือ O(K) สำหรับการคำนวณอาคารใหม่ และเมื่อรวมการเรียงรายการย้ายซึ่งมีเกือบ K รายการ ต้นทุนรวมเป็น O(K log K) (หัวข้อ 6.8)

บทที่ 4
กติกาการคำนวณ

หัวข้อนี้อธิบายกติกาที่ระบบใช้คำนวณตำแหน่งบนวงแหวน การเลือกอาคาร หมายเลขห้อง และการนับแขกที่ย้าย

4.1 ฟังก์ชันแฮชและตำแหน่งบนวงแหวน

วงแหวนมีตำแหน่งเป็นจำนวนเต็ม 0 ถึง $2^{64} - 1$ ระบบแปลงข้อความเป็นตำแหน่งด้วยฟังก์ชัน hash64 ตามขั้นตอนดังนี้

1. สร้างข้อความที่จะแฮช โดยแยกรูปแบบของแขกกับ virtual node ให้ชัดเจนและใช้รูปแบบเดิมทุกครั้ง

- แขกรหัส (c, s): guest:{c}:{s}
- virtual node ลำดับที่ j (j = 0 ถึง V − 1) ของอาคาร node_id: node:{node_id}:{j}

2. ถ้ามี salt ให้ต่อไว้หน้าข้อความในรูป salt|key (ถ้าไม่มี salt ใช้ข้อความเดิม) salt ใช้ในการทดลองเพื่อให้ได้ตำแหน่งชุดใหม่จากแขกและอาคารชุดเดิม

3. เข้ารหัสข้อความเป็นไบต์แบบ UTF-8 แล้วคำนวณ SHA-256 ได้ผลลัพธ์ 32 ไบต์

4. นำ 8 ไบต์แรก (64 บิต) แปลงเป็นจำนวนเต็มแบบ big-endian เป็นตำแหน่งบนวงแหวน

```python
def hash64(key: str, salt: str = "") -> int:
    text = key if salt == "" else f"{salt}|{key}"
    digest = hashlib.sha256(text.encode("utf-8")).digest()
    return int.from_bytes(digest[:8], "big")
```

รูปที่ 4.1 ฟังก์ชัน hash64

ตัวอย่างผลลัพธ์จริง (ไม่ใช้ salt)

ข้อความ	8 ไบต์แรกของ SHA-256 (hex)	ตำแหน่งบนวงแหวน
guest:1:1	2a7e853a775d4b50	3062031282815716176
guest:2:1	b04bbf21ab49333a	12703457325257143098
node:1:0	30a6adba2f06b0b1	3505680375118999729

ตารางที่ 4.1 ตัวอย่างตำแหน่งบนวงแหวนจาก hash64 (SHA-256 ตัดเหลือ 64 บิต ไม่ใช้ salt)

เหตุผลที่ไม่ใช้ hash() ของ Python ฟังก์ชัน hash() สุ่มค่าของสตริงใหม่ทุกครั้งที่เปิดโปรแกรม (hash randomization) ทำให้ตำแหน่งบนวงแหวนเปลี่ยนเมื่อปิดแล้วเปิดใหม่ ขัดกับข้อกำหนดที่ต้องได้ผลคงที่ข้ามการรัน SHA-256 ให้ค่าเดิมเสมอเมื่อข้อความและ salt เท่าเดิม และกระจายค่าได้ใกล้เคียงสม่ำเสมอ ซึ่งเป็นสมมติฐานของค่าคาดหวังการย้ายในหัวข้อ 4.5

4.2 การเลือกอาคารบนวงแหวนและการวนกลับ

โครงสร้างข้อมูลของ HashRing

ตัวแปร	ชนิด	หน้าที่
__points	list ของ (pos, node_id, j) เรียงจากน้อยไปมาก	เก็บทุกจุดของวงแหวน (M = N × V จุด)
__positions	list ของ pos ลำดับเดียวกับ __points	ใช้เป็นอาร์เรย์สำหรับ bisect_left
__nodes	set ของ node_id	ตรวจอาคารซ้ำหรืออาคารที่ไม่มีอยู่ในเวลา O(1)

ตารางที่ 4.2 โครงสร้างข้อมูลที่ใช้เก็บวงแหวนในคลาส HashRing

กติกาเลือกอาคาร แขกอยู่ที่อาคารเจ้าของ "จุดแรกที่มีตำแหน่งมากกว่าหรือเท่ากับตำแหน่งของแขก" ถ้าไม่มีจุดดังกล่าว (แขกอยู่หลังจุดสุดท้าย) ให้วนกลับไปจุดแรกของวงแหวน

```
def locate(self, position: int) -> int:
    if not self.__points:
        raise InvalidInputError("วงแหวนยังไม่มีอาคาร")
    i = bisect_left(self.__positions, position)
    if i == len(self.__positions):
        i = 0
    return self.__points[i][1]
```

รูปที่ 4.2 ฟังก์ชันเลือกอาคาร

bisect_left คืนดัชนีแรกที่ positions[i] >= position จึงตรงกับเงื่อนไข "มากกว่าหรือเท่ากับ" แขกที่ตำแหน่งตรงกับจุดอาคารพอดีจึงได้อาคารนั้น การเลือกอาคารคำนวณจากวงแหวนปัจจุบันทุกครั้ง (node_for) ไม่ใช้ตารางคำตอบที่บันทึกไว้ล่วงหน้า ขั้นตอนหาจุดมีต้นทุน O(log M)

ตัวอย่างวงแหวนขนาดเล็ก (วงแหวนขนาด 100, V = 1, กำหนดตำแหน่งเองเพื่อตรวจตรรกะ ไม่ใช่ผลจาก SHA-256 อาคาร A ที่ 20, B ที่ 50, C ที่ 80)

ตำแหน่งแขก	เริ่มต้น	เพิ่ม D ที่ 35	ลบ B
10	A	A	A
20	A	A	A
25	B	D	C
35	B	D	C
50	B	B	C
70	C	C	C
90	A(วนกลับ)	A	A

ตารางที่ 4.3 ผลการเลือกอาคารบนวงแหวนตัวอย่าง (ขนาด 100, V = 1) เมื่อเริ่มต้น เพิ่มอาคาร D และลบอาคาร B

ตำแหน่ง 90 อยู่หลังจุดสุดท้ายจึงวนกลับไปอาคาร A ตำแหน่ง 20 และ 50 ตรงจุดอาคารพอดีจึงได้อาคารนั้น เมื่อเพิ่ม D มีแขกย้าย 2 คน (B $\rightarrow$ D ทั้งคู่) และเมื่อลบ B มีแขกย้าย 3 คน (ทั้งหมดออกจาก B) ตรงกับค่าที่โจทย์กำหนด

4.3 กรณีจุดแฮชชนกัน

ระบบเก็บ ทุกจุด ไว้ในวงแหวน ไม่ทิ้งจุดที่ตำแหน่งซ้ำกัน และเรียงด้วยคีย์ (pos, node_id, j) โดยใช้การเรียง tuple ของ Python ซึ่งเทียบ pos ก่อน ถ้าเท่ากันเทียบ node_id แล้วจึงเทียบ j ตรงตามกติกาของโจทย์โดยไม่ต้องเขียนตรรกะเพิ่ม เมื่อแขกอยู่ที่ตำแหน่งที่มีหลายจุดซ้อนกัน bisect_left คืนจุดแรกตามลำดับนี้ เช่น อาคาร 1 และอาคาร 2 มีจุดที่ตำแหน่ง 50 เหมือนกัน แขกที่ตำแหน่ง 50 ได้อาคาร 1

- แขกหลายคนแฮชได้ตำแหน่งเดียวกัน และยังได้เลขห้องต่างกัน เพราะเลขห้องคำนวณจากรหัส (c, s) ไม่ใช่จากตำแหน่งแฮช
- ถ้าลบอาคารหนึ่งในคู่ที่ชนกัน จุดของอีกอาคารยังอยู่และรับแขกต่อได้ตามปกติ

4.4 หมายเลขห้องด้วย Cantor pairing function

หมายเลขห้องคำนวณจากรหัสแขกเท่านั้น โดยกำหนด a = c − 1 และ b = s − 1

$$\text{room_no} = ((a + b) \times (a + b + 1)) \,//\, 2 + b + 1$$

คำนวณด้วยจำนวนเต็มตลอด (// คือการหารจำนวนเต็ม) ไม่ใช้ทศนิยม และไม่มีส่วนใดของโปรแกรมใช้วิธี "เลขห้องล่าสุด + 1"

ตัวอย่างเลขห้อง

(c, s)	(1,1)	(2,1)	(1,2)	(3,1)	(2,2)	(1,3)	(10,10)
room_no	1	2	3	4	5	6	181

ตารางที่ 4.4 ตัวอย่างหมายเลขห้องจากสูตร Cantor pairing function

พิสูจน์ว่าเลขห้องไม่ซ้ำกัน ให้ t = a + b และ T(t) = t(t + 1)/2 จะได้ room_no = T(t) + b + 1

1. สำหรับ t คงที่ b มีค่า 0, 1, ..., t เลขห้องจึงเป็นจำนวนเต็มต่อเนื่อง T(t) + 1 ถึง T(t) + t + 1 = T(t + 1) รวม t + 1 ค่า และแต่ละ b ให้ค่าไม่ซ้ำกัน
2. แนวทแยงถัดไป (t + 1) เริ่มที่ T(t + 1) + 1 ซึ่งมากกว่าค่าสุดท้ายของแนวทแยง t ช่วงของแต่ละ t จึงไม่ทับกัน
3. ดังนั้นคู่ (a, b) ที่ต่างกันให้ room_no ต่างกันเสมอ และจำนวนเต็มบวกทุกค่ามีคู่ (c, s) เพียงคู่เดียวที่ให้ค่านั้น

ผลต่อการย้ายอาคาร เลขห้องไม่ขึ้นกับจำนวนอาคารหรือตำแหน่งบนวงแหวน เมื่อแขกย้ายอาคาร เปลี่ยนเฉพาะ node_id ส่วน room_no คงเดิม และเพราะ room_no ไม่ซ้ำกันทั้งระบบ ที่อยู่ห้อง (node_id, room_no) จึงไม่ซ้ำกันเช่นกัน การเพิ่มแขกใหม่ไม่กระทบที่อยู่ห้องของแขกเดิม

4.5 การนับการย้ายและอัตราการย้าย

การเพิ่ม/ลบอาคาร การเพิ่มอาคารเพิ่มเฉพาะ virtual node V จุดของอาคารใหม่ การลบอาคารลบทุกจุดของอาคารนั้น จุดของอาคารอื่นไม่เปลี่ยนรหัสหรือตำแหน่ง และ V คงที่ตลอดอายุของวงแหวน

ทำไมผู้ย้ายจึงเป็นไปตามกติกา

- เมื่อเพิ่มอาคาร จุดเดิมทั้งหมดอยู่ที่เดิม มีเพียงจุดใหม่แทรกเข้ามา แขกจะเปลี่ยนเจ้าของได้ก็ต่อเมื่อจุดใหม่กลายเป็นจุดแรกที่พบ ผู้ย้ายจึงย้ายเข้าอาคารใหม่เท่านั้น
- เมื่อลบอาคาร แขกที่จุดแรกที่พบไม่ใช่จุดของอาคารที่ลบ ยังพบจุดเดิม จึงไม่ย้าย มีเฉพาะแขกของอาคารที่ลบที่ต้องย้าย

วิธีหาผู้ย้าย ระบบใช้ วิธีตรวจใหม่ทุกคน ฟังก์ชัน __find_moves วนแขกทั้ง K คน คำนวณอาคารใหม่ด้วย node_for(c, s) จากวงแหวนหลังเปลี่ยน แล้วเทียบกับอาคารเดิม คนที่ต่างกันคือผู้ย้าย นับคนละหนึ่งครั้ง และสร้างรายการ Move(c, s, old_node, new_node, room_no) เรียงตาม (c, s) ต้นทุนของขั้นตอนนี้คือ O(K log M)

การรักษาข้อมูลให้สอดคล้อง หลังหาผู้ย้ายแล้ว __apply_moves ปรับข้อมูล 3 ชุดพร้อมกัน ได้แก่ __location (รหัสแขก → อาคาร), __room_index ((อาคาร, เลขห้อง) → รหัสแขก) และ __load (จำนวนแขกต่ออาคาร) โดยลบรายการ (old_node, room_no) ออกจาก __room_index ก่อน แล้วจึงเพิ่ม (new_node, room_no) ทำให้การค้นหาด้วยที่อยู่ห้องหลังย้ายยังให้คำตอบถูกต้อง ตรวจด้วย check_integrity และชุดทดสอบ test_search_still_correct_after_migration

การรายงาน MigrationReport เก็บรายการย้ายเป็น tuple และ moved_count = len(moves) จำนวนที่รายงานจึงเท่ากับรายการที่พิมพ์และส่งออกจริงเสมอ

- อัตราการย้าย = จำนวนแขกที่ย้าย / K (K คือจำนวนแขกก่อนคำสั่ง)
- ถ้า K = 0 รายงานจำนวนย้ายเป็น 0 และระบุว่าไม่คำนวณอัตราเพราะไม่มีแขก (rate = None)

ค่าคาดหวังและผลที่วัดได้ เมื่อแฮชกระจายสม่ำเสมอ การเพิ่มจาก N เป็น N + 1 อาคารมีผู้ย้ายคาดหวังราว K/(N + 1) และการลบ 1 จาก N อาคารราว K/N ตารางต่อไปนี้เป็นผลการทดลอง B (K = 10,000, V = 32, 10 ช่องทาง, ค่ามัธยฐานของ 3 รอบที่ salt 0, 1, 2) วงเล็บคือช่วงต่ำสุด–สูงสุดของ 3 รอบ

N	เพิ่มอาคาร: ย้าย (คาดหวัง K/(N+1))	ลบอาคาร: ย้าย (คาดหวัง K/N)	hash mod N: เพิ่ม / ลบ
5	1,347 = 13.47% (1,166–1,996) (คาดหวัง 1,667)	2,022 = 20.22% (1,481–2,127) (คาดหวัง 2,000)	8,365 (83.65%) / 8,010 (80.10%)
10	910 = 9.10% (719–1,011) (คาดหวัง 909)	864 = 8.64% (839–1,286) (คาดหวัง 1,000)	9,063 (90.63%) / 9,028 (90.28%)
20	456 = 4.56% (392–499) (คาดหวัง 476)	453 = 4.53% (369–481) (คาดหวัง 500)	9,551 (95.51%) / 9,507 (95.07%)

ตารางที่ 4.5 จำนวนแขกที่ย้ายเมื่อเพิ่ม/ลบอาคาร เทียบ Consistent Hashing กับ hash mod N และค่าคาดหวัง (K = 10,000, V = 32, มัธยฐานของ 3 รอบ, วงเล็บคือค่าต่ำสุด–สูงสุด)

บทที่ 5
ส่วนติดต่อผู้ใช้และไฟล์

5.1 เมนูข้อความ

โปรแกรมใช้งานผ่านเมนูข้อความใน terminal เรียกด้วยคำสั่ง python main.py เมนูทั้งหมดแสดงในตารางที่ 5.1

เมนู	การทำงาน
1	เริ่มระบบ: จำนวนอาคาร N, ค่า V, จำนวนช่องทาง, จำนวนแขกแต่ละช่องทาง, salt
2	เพิ่มแขกรายคน (c, s) หรือเป็นกลุ่ม (c, s_start, จำนวน)
3	นำแขกออกด้วย (c, s)
4/5	เพิ่ม / ลบอาคาร พร้อมรายงานจำนวน อัตรา และรายการแขกที่ย้าย
6/7	ค้นด้วยรหัสแขก $\rightarrow$ (node_id, room_no) / ค้นด้วย (node_id, room_no) $\rightarrow$ รหัสแขก
8	แสดงรายการห้องที่มีผู้พัก เรียงตาม node_id แล้วตาม room_no
9	รายงานความสมดุลของโหลด: ทุกอาคาร (รวมที่มี 0 คน), สูงสุด, ต่ำสุด, เฉลี่ย, SD, CV
10	วัดเวลาและหน่วยความจำของฟังก์ชันหลัก
11	เทียบ Consistent Hashing กับ hash mod N ด้วยชุดแขกและ salt เดียวกัน
12	ส่งออก CSV: แขกทั้งหมด, รายการย้ายล่าสุด, ผลวัดจากเมนู 10–11
0	ออกจากโปรแกรม

ตารางที่ 5.1 เมนูของโปรแกรม

```
เลือกเมนู: 1
จำนวนอาคาร N (>= 1): 10
virtual node ต่ออาคาร V (>= 1) [Enter = 32]:
จำนวนช่องทาง (>= 0) [Enter = 10]:
จำนวนแขกต่อช่องทาง (ตัวเลขเดียว = ใช้ทุกช่อง หรือพิมพ์ 10 ตัวคั่นด้วยเว้นวรรค): 100
salt [Enter = ไม่ใช้]:
เริ่มระบบแล้ว: N=10, V=32, ช่องทาง=10, แขกทั้งหมด K = 1000
```

รูปที่ 5.1 เมนูหลักของโปรแกรม และการเริ่มระบบด้วย N = 10, V = 32, แขก 10 ช่องทาง ช่องทางละ 100 คน

เมนู 10 และ 11 วัดผลบนระบบทดลองที่สร้างแยกต่างหาก จึงไม่กระทบระบบที่กำลังใช้งาน และวัดเพียง 1 รอบ ส่วนการทดลองเต็มรูปแบบ 3 รอบ ใช้สคริปต์ python -m experiments.run_experiments

5.2 ตัวอย่างการทำงาน

ตัวอย่างในหัวข้อนี้ต่อจากระบบที่เริ่มไว้ในรูปที่ 5.1 คือ N = 10, V = 32 และแขก 10 ช่องทาง ช่องทางละ 100 คน (K = 1,000) โดยไม่ใช้ salt

```
เลือกเมนู: 9
แขกทั้งหมด K = 1000 | อาคาร 10 หลัง
  อาคาร   1: 119 คน
  อาคาร   2: 135 คน
  อาคาร   3: 75 คน
  อาคาร   4: 148 คน
  อาคาร   5: 110 คน
  อาคาร   6: 98 คน
  อาคาร   7: 78 คน
  อาคาร   8: 92 คน
  อาคาร   9: 80 คน
  อาคาร  10: 65 คน
สูงสุด = 148 | ต่ำสุด = 65 | เฉลี่ย = 100.00 | SD = 26.06
CV = SD / เฉลี่ย = 0.2606 (ยิ่งต่ำยิ่งสมดุล)
```

รูปที่ 5.2 รายงานความสมดุลของโหลด (เมนู 9) เมื่อมี 10 อาคาร และแขก 1,000 คน

รายงานแสดงจำนวนแขกของทุกอาคาร อาคารที่มีแขกมากที่สุดมี 148 คน น้อยที่สุดมี 65 คน ค่าเฉลี่ยอยู่ที่ 100.00 คน ส่วนเบี่ยงเบนมาตรฐาน 26.06 คิดเป็น CV = 0.2606

การเพิ่มอาคารและรายการแขกที่ย้าย

```
เลือกเมนู: 4
รหัสอาคารใหม่ (>= 1) [Enter = 11]:
=== เพิ่มอาคาร 11 ===
แขกก่อนคำสั่ง K = 1000
จำนวนแขกที่ย้าย = 88
อัตราการย้าย = 88/1000 = 0.0880 (8.80%)
มีรายการย้าย 88 รายการ แสดงทั้งหมดหรือไม่ (ไม่ = แสดง 50 รายการแรก) (y/N): n
รหัสแขก (c, s)     อาคารเดิม อาคารใหม่          ห้อง
(1, 4)                   8        11            10
(1, 11)                  1        11            66
(1, 15)                  2        11           120
(1, 19)                  3        11           190
(1, 23)                  3        11           276
(1, 26)                 10        11           351
(1, 29)                  5        11           435
(1, 34)                  3        11           595
(1, 41)                  1        11           861
(1, 43)                  3        11           946
(1, 53)                 10        11          1431
(1, 74)                  8        11          2775
(1, 78)                  1        11          3081
(1, 84)                  3        11          3570
(1, 92)                  7        11          4278
```

รูปที่ 5.3 การเพิ่มอาคาร 11 (เมนู 4) และรายการแขกที่ย้าย (แสดงบางส่วน)

เมื่อเพิ่มอาคาร 11 มีแขกย้าย 88 คนจาก 1,000 คน คิดเป็นอัตราการย้าย 8.80% ใกล้กับค่าคาดหวังทางทฤษฎี K/(N + 1) = 1,000/11 ≈ 90.9 คน แขกที่ย้ายทั้งหมดย้ายเข้าอาคาร 11 ซึ่งเป็นอาคารใหม่ และหมายเลขห้องของแต่ละคนยังเป็นเลขเดิม แขกอีก 912 คนที่ไม่ได้ย้ายยังอยู่ที่เดิมทุกคน รายการย้ายทั้ง 88 รายการส่งออกเป็นไฟล์ moves.csv ได้ (หัวข้อ 5.4)

การค้นหาสองทาง

```
เลือกเมนู: 6
ช่องทาง c (>= 1): 1
ลำดับ s (>= 1): 4
แขก (1, 4) -> อาคาร 11, ห้อง 10
```

```
เลือกเมนู: 7
รหัสอาคาร node_id (>= 1): 11
หมายเลขห้อง room_no (>= 1): 10
อาคาร 11 ห้อง 10 -> แขก (1, 4)
```

รูปที่ 5.4 การค้นด้วยรหัสแขก (เมนู 6) และการค้นด้วยที่อยู่ห้อง (เมนู 7)

แขก (1, 4) เป็นหนึ่งในแขกที่ย้ายจากอาคาร 8 ไปอาคาร 11 ในรูปที่ 5.3 เมื่อค้นด้วยรหัสแขก (1, 4) ได้อาคาร 11 ห้อง 10 และเมื่อค้นกลับด้วยที่อยู่ (11, 10) ได้แขก (1, 4) แสดงว่าดัชนีทั้งสองทางถูกปรับตามการย้ายครบแล้ว

5.3 การจัดการข้อมูลนำเข้าที่ผิดและกรณีขอบ

โปรแกรมแบ่งข้อผิดพลาดเป็น 2 ชนิด ชนิดแรกคือผู้ใช้พิมพ์ค่าผิดรูปแบบ เช่น พิมพ์ตัวอักษรแทนตัวเลข ใส่เลขติดลบ หรือเรียกเมนูอื่นก่อนเริ่มระบบ โปรแกรมจะแสดงข้อความขึ้นต้นด้วย “[!] ข้อมูลไม่ถูกต้อง” ชนิดที่สองคือคำสั่งที่ขัดกับกติกาของระบบ เช่น เพิ่มรหัสแขกซ้ำ หรือลบอาคารที่ไม่มีอยู่ HotelSystem จะแจ้งข้อผิดพลาดในกลุ่ม HotelError และเมนูแสดงข้อความ “[!] ปฏิเสธ: ... (ข้อมูลเดิมไม่เปลี่ยน)”
ทั้งสองกรณี โปรแกรมกลับไปที่เมนูหลักโดยไม่หยุดทำงาน และข้อมูลในระบบไม่เปลี่ยนแปลง ถ้าการเริ่มระบบใหม่ล้มเหลว ระบบเดิมยังอยู่ครบ นอกจากนี้เมนูยังดักข้อผิดพลาดที่ไม่คาดคิดไว้อีกชั้นหนึ่ง เพื่อไม่ให้โปรแกรมหยุดทำงานกลางคัน

กรณี	ตัวอย่างคำสั่ง	ข้อความที่โปรแกรมแสดง
เพิ่มรหัสแขกที่มีอยู่แล้ว	เพิ่มแขก (1, 1)	ปฏิเสธ: รหัสแขก (1, 1) มีอยู่แล้ว
เพิ่มแขกเป็นกลุ่มที่ทับรหัสเดิมบางส่วน	เพิ่มช่องทาง 1 ลำดับ 98–102	ปฏิเสธ: รหัสแขกซ้ำในกลุ่มที่จะเพิ่ม: [(1, 98), (1, 99), (1, 100)] — ปฏิเสธทั้งกลุ่ม
นำแขกที่ไม่มีอยู่ออก	นำแขก (99, 99) ออก	ปฏิเสธ: ไม่พบรหัสแขก (99, 99)
ลบอาคารที่ไม่มีอยู่	ลบอาคาร 99	ปฏิเสธ: ไม่พบอาคาร 99
เพิ่มรหัสอาคารที่มีอยู่แล้ว	เพิ่มอาคาร 1	ปฏิเสธ: อาคาร 1 มีอยู่แล้ว วงแหวนไม่เปลี่ยน
ค้นด้วยรหัสแขกแล้วไม่พบ	ค้นแขก (99, 99)	ไม่พบแขก (99, 99) ในระบบ
ค้นด้วยที่อยู่ห้องแล้วไม่พบ	ค้นอาคาร 1 ห้อง 999999	ไม่พบผู้พักที่อาคาร 1 ห้อง 999999
พิมพ์ตัวอักษรแทนตัวเลข	พิมพ์ x เป็นช่องทาง	ข้อมูลไม่ถูกต้อง: ‘x’ ไม่ใช่จำนวนเต็ม

ตารางที่ 5.2 กรณีขอบและข้อความที่โปรแกรมแสดง

ในกรณีเพิ่มแขกเป็นกลุ่ม แขก 5 คนในกลุ่มมีรหัสซ้ำกับแขกเดิมเพียง 3 คน แต่โปรแกรมปฏิเสธทั้งกลุ่ม และไม่เพิ่มแขก (1, 101) กับ (1, 102) ที่ไม่ซ้ำ ข้อมูลจึงไม่ค้างอยู่ครึ่งทาง ส่วนกรณีที่ต้องเตรียมสถานะพิเศษ เช่น การลบอาคารสุดท้าย และระบบที่ไม่มีแขกเลย (K = 0) ตรวจด้วยสคริปต์ทดสอบอัตโนมัติ

```
เลือกเมนู: 2
1) รายคน  2) เป็นกลุ่ม : 1
ช่องทาง c (>= 1): 1
ลำดับ s (>= 1): 1
[!] ปฏิเสธ: รหัสแขก (1, 1) มีอยู่แล้ว (ข้อมูลเดิมไม่เปลี่ยน)
```

```
เลือกเมนู: 2
1) รายคน  2) เป็นกลุ่ม : 2
ช่องทาง c (>= 1): 1
ลำดับเริ่มต้น s_start (>= 1): 98
จำนวนแขก (>= 0): 5
[!] ปฏิเสธ: รหัสแขกซ้ำในกลุ่มที่จะเพิ่ม: [(1, 98), (1, 99), (1, 100)] – ปฏิเสธทั้งกลุ่ม (ข้อมูลเดิมไม่เปลี่ยน)
```

```
เลือกเมนู: 3
ช่องทาง c (>= 1): 99
ลำดับ s (>= 1): 99
[!] ปฏิเสธ: ไม่พบรหัสแขก (99, 99) (ข้อมูลเดิมไม่เปลี่ยน)
```

```
เลือกเมนู: 5
อาคารที่มีอยู่: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11
รหัสอาคารที่จะลบ: 99
[!] ปฏิเสธ: ไม่พบอาคาร 99 (ข้อมูลเดิมไม่เปลี่ยน)
```

```
เลือกเมนู: 4
รหัสอาคารใหม่ (>= 1) [Enter = 12]: 1
[!] ปฏิเสธ: อาคาร 1 มีอยู่แล้ว วงแหวนไม่เปลี่ยน (ข้อมูลเดิมไม่เปลี่ยน)
```

```
เลือกเมนู: 6
ช่องทาง c (>= 1): 99
ลำดับ s (>= 1): 99
ไม่พบแขก (99, 99) ในระบบ
```

```
เลือกเมนู: 7
รหัสอาคาร node_id (>= 1): 1
หมายเลขห้อง room_no (>= 1): 999999
ไม่พบผู้พักที่อาคาร 1 ห้อง 999999
```

```
เลือกเมนู: 6
ช่องทาง c (>= 1): x
[!] ข้อมูลไม่ถูกต้อง: 'x' ไม่ใช่จำนวนเต็ม
```

รูปที่ 5.5 ตัวอย่างกรณีขอบที่โปรแกรมปฏิเสธ

5.4 การส่งออกไฟล์ CSV

ข้อมูลในระบบส่งออกเป็นไฟล์ CSV ได้ผ่านเมนู 12 ทุกไฟล์เขียนด้วยการเข้ารหัส UTF-8 แถวแรกเป็นชื่อคอลัมน์ และเปิดไฟล์แบบ newline="" เพื่อไม่ให้มีบรรทัดว่างแทรกเมื่อเปิดบน Windows

ไฟล์	สร้างจาก	คอลัมน์	จำนวนแถวข้อมูล
data/guests.csv	เมนู 12 → 1	channel_id, sequence_id, node_id, room_no	เท่ากับจำนวนแขกในระบบ (K) เรียงตาม node_id แล้วตาม room_no
data/moves.csv	เมนู 12 → 2	channel_id, sequence_id, old_node_id, new_node_id, room_no	เท่ากับจำนวนแขกที่ย้ายในคำสั่งเพิ่มหรือลบอาคารครั้งล่าสุด
data/experiment_menu.csv	เมนู 12 → 3	experiment, salt, method, K, N, V, metric, value, unit	หนึ่งแถวต่อหนึ่งค่าที่วัดจากเมนู 10 หรือ 11
experiments/results/experiment_A.csv, experiment_B.csv, experiment_C.csv	สคริปต์การทดลอง	experiment, salt, method, K, N, V, metric, value, unit	ผลของทุกรอบ ทุก salt
experiments/results/summary.csv	สคริปต์การทดลอง	experiment, method, K, N, V, metric, unit, rounds, median, mean, min, max	ผลสรุปของ 3 รอบ

ตารางที่ 5.3 การส่งออกไฟล์ CSV

หลังส่งออกไฟล์แขก โปรแกรมเทียบจำนวนแถวที่เขียนกับจำนวนแขกในระบบให้ทันที และหลังส่งออกรายการย้าย โปรแกรมแสดงจำนวนแถวคู่กับจำนวนแขกที่รายงานว่าย้าย ผู้ใช้จึงตรวจได้ทันทีว่าไฟล์ครบ

```
เลือกเมนู: 12
ส่งออก: 1) แขกทั้งหมด  2) รายการย้ายล่าสุด  3) ผลวัด/เทียบจากเมนู 10-11
เลือก: 1
ชื่อไฟล์ [Enter = data\guests.csv]:
เขียน 1000 แถว -> data\guests.csv (K = 1000: ตรงกับจำนวนแขกในระบบ)
```

รูปที่ 5.6 การส่งออกไฟล์แขกและไฟล์รายการย้าย (เมนู 12)

ไฟล์ตัวอย่างที่ส่งมาพร้อมโปรแกรมได้จากกรณีในหัวข้อ 5.2 ไฟล์ guests.csv มีแขก 1,000 แถว ส่งออกหลังเพิ่มอาคาร 11 จึงมีแขก 88 แถวที่อยู่อาคาร 11 ส่วน moves.csv มี 88 แถว ตรงกับจำนวนแขกที่ย้ายในรูปที่ 5.3

บทที่ 6
การวิเคราะห์ BigO

6.1 วัตถุประสงค์และขอบเขต

หัวข้อนี้วิเคราะห์ต้นทุนเชิงเวลาและเชิงพื้นที่ของการดำเนินการหลักของระบบโรงแรม โดยแบ่งเป็น 6 กลุ่ม ได้แก่ (1) การสร้างวงแหวน (2) การจัดแขก (3) การคำนวณอาคารจากวงแหวน (4) การค้นหา (5) การเรียงรายการห้อง และ (6) การเพิ่มและลบอาคาร แต่ละกลุ่มแยกวิเคราะห์ต้นทุนทีละขั้นตอน แล้วรวมเป็นความซับซ้อนของกลุ่ม ผลการวิเคราะห์นี้ใช้เปรียบเทียบกับผลการทดลองในหัวข้อ 7 และสรุปเปรียบเทียบในหัวข้อ 7.5

6.2 สัญลักษณ์ที่ใช้

สัญลักษณ์	ความหมาย
K	จำนวนแขกในระบบ
N	จำนวนอาคาร
V	จำนวน virtual node ต่ออาคาร
M = N·V	จำนวนจุดทั้งหมดบนวงแหวน
B	จำนวนแขกที่ต้องย้ายเมื่อเพิ่มหรือลบอาคาร (B ≤ K)

ตารางที่ 6.1 สัญลักษณ์และความหมาย

6.3 กลุ่มที่ 1: การสร้างวงแหวน

การสร้างวงแหวนเรียก add_node ทีละอาคาร รวม N ครั้ง แต่ละครั้งคำนวณแฮชของจุดอาคารใหม่ V จุด ต่อท้าย list แล้วเรียง list ทั้งหมดใหม่ และสร้าง __positions ใหม่ ต้นทุนของวงแหวนจึงขึ้นกับ N และ V เท่านั้น ไม่ขึ้นกับจำนวนแขก

ขั้นตอน (add_node ครั้งที่ i, มี iV จุด)	ต้นทุนเวลา
แฮช V จุดของอาคารใหม่	O(V)
ต่อท้ายแล้วเรียง list ทั้งหมด	O(iV log iV) กรณีเลวร้ายสุด (Timsort บน list ที่เกือบเรียงแล้ว ≈ O(iV + V log V))
สร้าง __positions ใหม่	O(iV)
รวม N ครั้ง	O(N · M log M) กรณีเลวร้ายสุด, ทำงานจริงใกล้ $O(N^2V)$

ตารางที่ 6.2 ต้นทุนของการสร้างวงแหวน

Timsort ของ Python ทำงานเร็วกับ list ที่เกือบเรียงแล้ว แต่ละครั้งจึงใช้เวลาจริงใกล้ O(iV + V log V) รวม N ครั้งใกล้ O(N^2V) ผลวัดในการทดลอง A (N = 10, V = 32) ใช้เวลาเพียง 0.44–0.46 ms และไม่เปลี่ยนตาม K พื้นที่ที่ใช้คือ O(NV) สำหรับเก็บจุดบนวงแหวน

6.4 กลุ่มที่ 2: การจัดแขก

แขกแต่ละคนถูกจัดเข้าอาคารโดยคำนวณแฮชของรหัสแขก ค้นหาจุดแรกบนวงแหวนที่ค่าแฮชไม่น้อยกว่าด้วย bisect (วนกลับไปจุดแรกหากเกินจุดสุดท้าย) แล้วบันทึกแขกลงอาคารพร้อมเลขห้อง และลงดัชนีรหัสแขก ขั้นที่แพงที่สุดต่อคนคือการค้นบนวงแหวน

ขั้นตอน	ต้นทุนเวลา
ตรวจรหัสซ้ำในดัชนี	O(1) เฉลี่ย
คำนวณแฮชของรหัสแขก	O(1)
ค้นอาคารเจ้าของด้วย bisect	O(log NV)
กำหนดเลขห้องและบันทึกลงอาคารและดัชนี	O(1) เฉลี่ย
รวมต่อคน	O(log NV)

ตารางที่ 6.3 ต้นทุนของการจัดแขก 1 คน

การเพิ่มแขก g คนใช้เวลา O(g log NV) ดังนั้นการจัดแขกทั้งหมด K คนใช้เวลา O(K log NV) พื้นที่เพิ่มคือ O(g) การนำแขกออก 1 คน (remove_guest) ไม่ต้องค้นวงแหวน ขั้นตอนคือ ตรวจรหัส O(1) ดึงอาคารออกจาก __location O(1) เฉลี่ย คำนวณเลขห้อง O(1) ลบรายการออกจาก __room_index และลด __load O(1) เฉลี่ย รวมเป็น O(1) เฉลี่ย

6.5 กลุ่มที่ 3: การคำนวณอาคารจากวงแหวน

ฟังก์ชันนี้ตอบว่าแขกรหัสหนึ่งควรอยู่อาคารใด โดยไม่ต้องเปิดดูข้อมูลแขกทั้งหมด จึงใช้เฉพาะการแฮชและการค้นบนวงแหวน

ขั้นตอน	ต้นทุนเวลา
คำนวณแฮชของรหัสแขก	O(1)
ค้นจุดบนวงแหวนด้วย bisect	O(log NV)
รวม	O(log NV)

ตารางที่ 6.4 ต้นทุนของการคำนวณอาคาร

ต้นทุนไม่ขึ้นกับ K และไม่ใช้พื้นที่เพิ่ม (O(1))

6.6 กลุ่มที่ 4: การค้นหา

ระบบค้นหาแขกได้สองทาง คือด้วยรหัสแขก และด้วยที่อยู่ห้อง (อาคารและเลขห้อง) ทั้งสองทางอาศัยตารางแฮชที่สร้างไว้ตอนจัดแขก จึงไม่ต้องไล่ดูแขกทีละคน

การค้นหา	ขั้นตอน	ต้นทุนเวลา
ค้นด้วยรหัสแขก (find_by_guest)	อ่าน __location[(c, s)] แล้วคำนวณเลขห้อง	O(1) เฉลี่ย
ค้นด้วยที่อยู่ห้อง (find_by_room)	อ่าน __room_index[(node_id, room_no)]	O(1) เฉลี่ย

ตารางที่ 6.5 ต้นทุนของการค้นหา

ทั้งสองทางเป็นค่าเฉลี่ยของ dict กรณีเลวร้ายสุดเมื่อคีย์ชนกันหมดเป็น O(K) ไม่ใช้พื้นที่เพิ่ม ถ้าไม่มีดัชนี __room_index การค้นด้วยที่อยู่ห้องต้องไล่ดูแขกทุกคน ซึ่งเป็น O(K)

6.7 กลุ่มที่ 5: การเรียงรายการห้อง

list_rooms สร้างรายการผลลัพธ์จาก __room_index แล้วเรียงตาม (node_id, room_no) เมนู 8 แสดงเพียงบางแถว แต่การเรียงทำกับแขกทั้ง K คนเสมอ และการส่งออก guests.csv ก็ใช้รายการนี้

ขั้นตอน	ต้นทุนเวลา
สร้างรายการผลลัพธ์ K แถวจาก __room_index	O(K)
เรียงด้วยคีย์ (node_id, room_no)	O(K log K) กรณีเลวร้ายสุด
รวม	O(K log K)

ตารางที่ 6.6 ต้นทุนของการเรียงรายการห้อง

พื้นที่เพิ่มคือ O(K) สำหรับลิสต์ผลลัพธ์

6.8 กลุ่มที่ 6: การเพิ่มและลบอาคาร

กลุ่มนี้เป็นจุดที่ Consistent Hashing ต่างจาก hash mod N มากที่สุด เมื่อเปลี่ยนจำนวนอาคาร ต้องตรวจว่าแขกคนใดมีเจ้าของใหม่ แล้วย้ายเฉพาะแขกเหล่านั้น ต้นทุนจึงประกอบด้วยการหาเจ้าของใหม่ของแขกทุกคน และการย้ายแขก B คน

ขั้นตอน	Consistent Hashing: เพิ่มอาคาร	Consistent Hashing: ลบอาคาร	hash mod N
(ก) ปรับวงแหวน / รายการอาคาร	แฮช V จุด O(V) + เรียง O(NV log NV) + สร้าง __positions O(NV) → O(NV log NV) กรณีเลวร้ายสุด	กรอง list O(NV) + สร้าง __positions O(NV) → O(NV)	ตรวจซ้ำและต่อท้าย / list.remove → O(N)
(ข) หาแขกที่ต้องย้าย	node_for K ครั้ง O(K log NV) + เรียงรายการย้าย O(B log B)	เหมือนเพิ่มอาคาร	node_for K ครั้ง O(K) + เรียง O(B log B) โดย B ≈ K → O(K log K)
(ค) ปรับข้อมูลและดัชนี	O(B) เฉลี่ย	O(B) เฉลี่ย	O(B) = O(K) เฉลี่ย
รวม	O(NV log NV + K log NV + B log B)	O(NV + K log NV + B log B)	O(K log K)

ตารางที่ 6.7 ต้นทุนของการเพิ่มและลบอาคาร

เมื่อฟังก์ชันแฮชกระจายสม่ำเสมอ จำนวนแขกที่ย้ายมีค่าคาดหวัง B ≈ K/(N+1) เมื่อเพิ่มอาคาร และ B ≈ K/N เมื่อลบอาคาร และแขกที่ย้ายต้องเกี่ยวข้องกับอาคารที่เพิ่มหรือถูกลบเท่านั้น

สำหรับ hash mod N การเปลี่ยนจำนวนอาคารทำให้ต้องคำนวณอาคารใหม่ให้แขกทุกคน (O(K)) และเกือบทุกคนได้อาคารใหม่ คือ B ≈ K·N/(N+1) เมื่อเพิ่ม และ B ≈ K(1 − 1/N) เมื่อลบ เมื่อรวมการเรียงรายการย้าย O(B log B) ต้นทุนรวมจึงเป็น O(K log K) ระดับความซับซ้อนใกล้เคียงกับ Consistent Hashing แต่จำนวนแขกที่ต้องย้ายต่างกันเป็นลำดับขนาด งานที่ทำต่อแขกที่ย้าย (สร้างรายการย้าย เรียง และปรับดัชนี) ของ hash mod N จึงมากกว่าประมาณ 10 เท่า ซึ่งอธิบายผลวัดในหัวข้อ 7.3 ที่ Consistent Hashing ใช้เวลา 11–13 ms ขณะที่ hash mod N ใช้ 17–19 ms พื้นที่เพิ่มของกลุ่มนี้คือ O(B) สำหรับรายการย้าย และ O(NV) สำหรับ list ใหม่ที่สร้างตอนปรับวงแหวน

6.9 พื้นที่รวมของระบบ

ส่วนประกอบ	พื้นที่
วงแหวน	O(NV)
ข้อมูลแขกและดัชนี (รหัสแขก, ห้อง)	O(K)
อาคาร	O(N)
รวม	O(K + NV)

ตารางที่ 6.8 พื้นที่ที่ใช้

พื้นที่โตเชิงเส้นตามจำนวนแขก ส่วนของ virtual node เป็นส่วนเล็กเมื่อ K มากกว่า NV มาก

6.10 สมมติฐานของการวิเคราะห์

1. นับจำนวนการดำเนินการพื้นฐาน ไม่รวมค่าคงที่และภาระของตัวแปลภาษา Python
2. การแฮชรหัสแขกและรหัสอาคารใช้เวลา O(1) เพราะรหัสสั้นและมีความยาวคงที่
3. ตารางแฮช (dict, set) ค้นและแทรกใช้ O(1) เฉลี่ย โดยไม่พิจารณากรณีแฮชชนกันหนัก
4. วงแหวนเก็บเป็นลิสต์เรียงตามค่าแฮช ค้นด้วย bisect จึงใช้ O(log NV)
5. ฟังก์ชันแฮชกระจายค่าสม่ำเสมอ แขกจึงกระจายตามขนาดช่วงบนวงแหวน และ B มีค่าคาดหวังตามที่ระบุ
6. การเรียงลำดับเป็น comparison sort (Timsort) ที่ O(K log K) และการเทียบคีย์แต่ละคู่ใช้ O(1)
7. ในการทดลอง K มากกว่า NV มาก (K ≥ 1,000 เทียบกับ NV = 320) จึงใช้ O(K log NV) แทนผลรวมที่มีพจน์ NV log NV ได้
8. ค่าที่ระบุเป็นกรณีเฉลี่ย ยกเว้นที่เขียนว่ากรณีเลวร้ายสุด ส่วน B เป็นค่าคาดหวัง
9. กลุ่มที่ 6 ถือว่าโปรแกรมตรวจเจ้าของใหม่ของแขกทุกคนทั้งตอนเพิ่มและตอนลบอาคาร ซึ่งสอดคล้องกับเวลาที่วัดได้ในหัวข้อ 7.3 (ดูหมายเหตุในหัวข้อนั้น)

บทที่ 7

การทดลอง

7.1 วิธีการวัด

การทดลองแบ่งเป็น 3 ชุด ชุด A เปลี่ยนจำนวนแขก K เพื่อดูการเติบโตของเวลาและหน่วยความจำ ชุด B เปลี่ยนจำนวนอาคาร N เพื่อเปรียบเทียบอัตราการย้ายแขกของ Consistent Hashing กับ hash mod N เมื่อเพิ่มหรือลบอาคาร 1 หลัง และชุด C เปลี่ยนจำนวน virtual node V เพื่อดูผลต่อความสมดุลของโหลดและหน่วยความจำ

ชุด	ตัวแปรที่เปลี่ยน	ค่าคงที่
A	K = 1,000 / 10,000 / 100,000	N = 10, V = 32
B	N = 5 / 10 / 20	K = 10,000, V = 32
C	V = 1 / 8 / 32	N = 10, K = 10,000

ตารางที่ 7.1 พารามิเตอร์ของการทดลอง

การวัดเวลาใช้ time.perf_counter() จับเวลาทีละขั้นตอน โดยไม่รวมเวลาแสดงผลและเขียนไฟล์ และปิด garbage collector ระหว่างจับเวลาเพื่อลดความผันผวน สำหรับการค้นหา ใช้ชุดรหัสแขกสุ่ม 1,000 รายการ (seed 12345) ซึ่งเหมือนกันทุกรอบและทุกวิธี ค้นทั้งชุดแล้วหารเป็นเวลาต่อครั้ง หน่วยความจำวัดด้วย tracemalloc โดยบันทึกส่วนที่เพิ่มขึ้นจากจุดเริ่มเดียวกัน (ค่า current และ peak หน่วย MiB) ซึ่งเป็นหน่วยความจำที่ Python ติดตาม ไม่ใช่หน่วยความจำของทั้งโปรเซส และวัดแยกรอบจากการจับเวลา เนื่องจาก tracemalloc ทำให้โปรแกรมช้าลง ข้อมูลแขกสร้างจาก 10 ช่องทาง โดยแต่ละช่องทางมีจำนวนแขกต่างกันไม่เกิน 1 คน

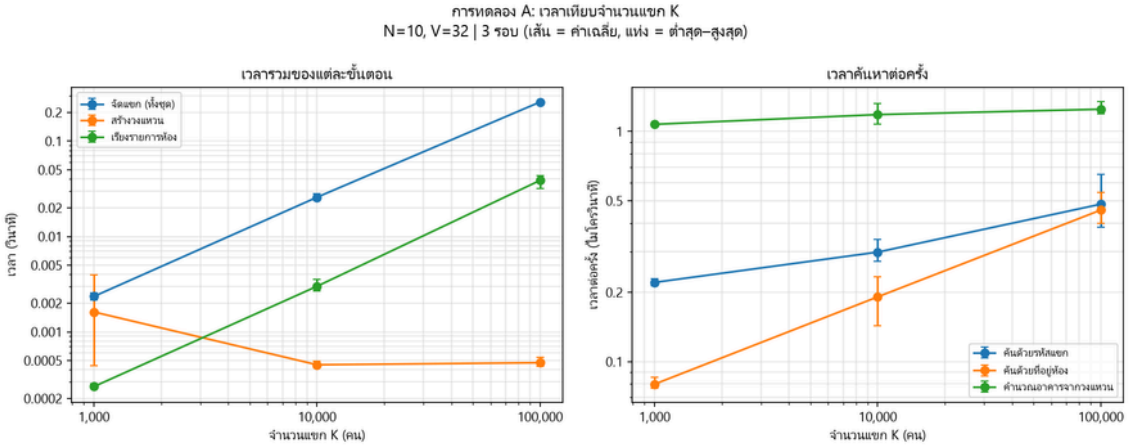
แต่ละการทดลองทำซ้ำ 3 รอบ ด้วยค่า salt ของฟังก์ชันแฮชเท่ากับ 0, 1 และ 2 ตารางรายงานค่ามัธยฐานของ 3 รอบ เพราะรอบแรกอาจช้าผิดปกติ (เช่นการสร้างวงแหวนที่ K = 1,000 รอบหนึ่งใช้ 3.95 ms ขณะที่อีกสองรอบใช้ประมาณ 0.44 ms) ส่วนกราฟแสดงค่าเฉลี่ยพร้อมช่วงต่ำสุดถึงสูงสุด ในชุด B ใช้รหัสแขก รหัสอาคาร และ salt ชุดเดียวกันทั้ง Consistent Hashing และ hash mod N ในรอบเดียวกัน และกำหนดว่าการเพิ่มอาคารใช้รหัส N+1 ส่วนการลบใช้รหัส N/2+1 (ปัดเศษลง) แต่ละกรณีเริ่มจากสถานะเดิมใหม่ ระหว่างทดลอง โปรแกรมตรวจความถูกต้องทุกรอบ หากจำนวนแขกไม่เท่า K หรือ Consistent Hashing ย้ายแขกผิดกติกา การทดลองจะหยุดทันที

7.2 ผลการทดลองชุด A: เวลาและหน่วยความจำเทียบจำนวนแขก K

ชุด A วัดเฉพาะ Consistent Hashing ที่ N = 10 และ V = 32 โดยเปลี่ยน K จาก 1,000 เป็น 10,000 และ 100,000

ตัวชี้วัด	K = 1,000	K = 10,000	K = 100,000
สร้างวงแหวน (ms)	0.461	0.443	0.445
จัดแขกทั้งชุด (ms)	2.363	25.35	259.4
จัดแขกต่อคน (µs)	2.36	2.54	2.59
เรียงรายการห้อง (ms)	0.259	2.737	40.85
ค้นด้วยรหัสแขก (µs/ครั้ง)	0.220	0.285	0.416
ค้นด้วยที่อยู่ห้อง (µs/ครั้ง)	0.078	0.196	0.423
คำนวณอาคารจากวงแหวน (µs/ครั้ง)	1.077	1.154	1.203
หน่วยความจำ current (MiB)	0.246	2.249	26.85
หน่วยความจำ peak (MiB)	0.259	2.378	28.70

ตารางที่ 7.2 ผลการทดลองชุด A (ค่ามัธยฐานของ 3 รอบ)



รูปที่ 7.1 กราฟเวลาเทียบจำนวนแขก K (N = 10, V = 32, 3 รอบ เส้นคือค่าเฉลี่ย แท่งคือช่วงต่ำสุดถึงสูงสุด) กราฟซ้าย แสดงเวลารวมของแต่ละขั้นตอน กราฟขวาแสดงเวลาค้นหาต่อครั้ง แกนทั้งสองเป็นสเกลลอการิทึม การอภิปรายผล เมื่อ K เพิ่มขึ้น 10 เท่า เวลาจัดแขกทั้งชุดเพิ่มขึ้นประมาณ 10.7 เท่า (จาก K = 1,000 เป็น 10,000) และ 10.2 เท่า (จาก 10,000 เป็น 100,000) เวลาต่อคนจึงคงที่ที่ประมาณ 2.4–2.6 µs แสดงว่าการจัดแขกโตเป็นเส้นตรงตาม K สอดคล้องกับการวิเคราะห์ O(K log NV) ที่ log NV คงที่ในการทดลองนี้ ส่วนการสร้าง

วงแหวนใช้เวลาราว 0.44–0.46 ms ไม่ว่า K จะเป็นเท่าใด เพราะขนาดของวงแหวนขึ้นกับ N·V = 320 จุดเท่านั้น การเรียงรายการห้องใช้เวลาเพิ่มขึ้น 10.6 เท่าและ 14.9 เท่าตามลำดับ ซึ่งสูงกว่าเส้นตรงเล็กน้อย ใกล้เคียงกับค่าคาดหวังของ K log K (13.3 เท่า และ 12.5 เท่า) การคำนวณอาคารจากวงแหวนใช้เวลาประมาณ 1.1–1.2 µs เพิ่มขึ้นเพียงราว 12% เมื่อ K เพิ่มขึ้น 100 เท่า จึงถือว่าไม่ขึ้นกับ K ตามที่วิเคราะห์ ส่วนการค้นด้วยรหัสแขกและด้วยที่อยู่ห้อง แม้ในทางทฤษฎีเป็น O(1) แต่เวลาที่วัดได้เพิ่มขึ้นตาม K (การค้นด้วยที่อยู่ห้องเพิ่มจาก 0.078 เป็น 0.423 µs) ทั้งที่ยังอยู่ในระดับต่ำกว่า 0.5 µs ซึ่งน่าจะเป็นผลของโครงสร้างข้อมูลที่ใหญ่ขึ้นจนใช้หน่วยความจำแคชของซีพียูได้น้อยลง แต่การทดลองนี้ไม่ได้วัดสาเหตุดังกล่าวโดยตรง จึงถือเป็นข้อสันนิษฐาน
หน่วยความจำที่ใช้โตตาม K ประมาณ 236–282 ไบต์ต่อแขก และค่า peak สูงกว่าค่า current ราว 6–8% ซึ่งสอดคล้องกับพื้นที่ O(K + NV)

7.3 ผลการทดลองชุด B: อัตราการย้ายเมื่อเพิ่มหรือลบอาคาร 1 หลัง

ชุด B เปรียบเทียบ Consistent Hashing (ring) กับ hash mod N ที่ K = 10,000 และ V = 32 โดยนิยามอัตราการย้ายเป็นจำนวนแขกที่ย้ายหารด้วย K

N ก่อนลบ	ring ย้าย (คน)	ring อัตรา	ring ช่วง min–max	ค่าคาดหวัง 1/N	mod N ย้าย (คน)	mod N อัตรา
5	2,022	0.202	0.148–0.213	0.200	8,010	0.801
10	864	0.086	0.084–0.129	0.100	9,028	0.903
20	453	0.045	0.037–0.048	0.050	9,507	0.951

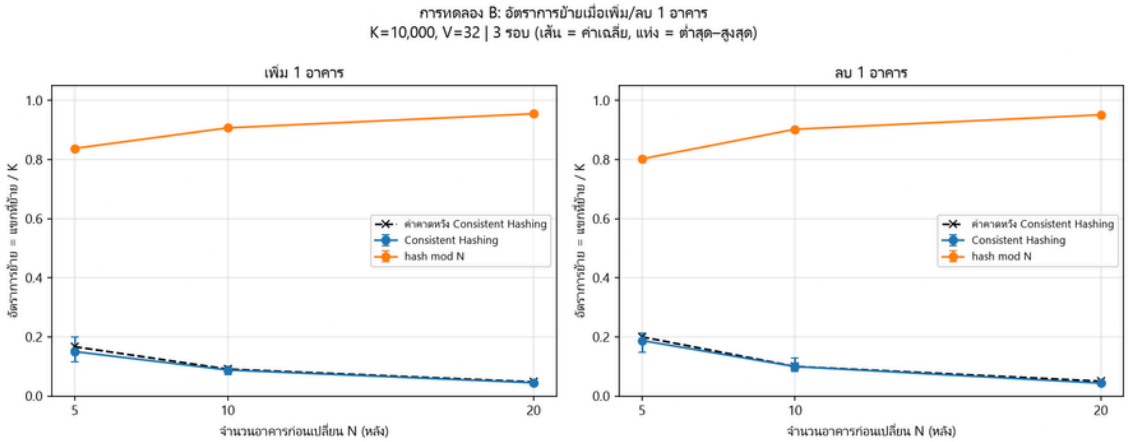
ตารางที่ 7.3 อัตราการย้ายเมื่อลบอาคาร 1 หลัง (ค่ามัธยฐาน)

N ก่อนเพิ่ม	ring ย้าย (คน)	ring อัตรา	ring ช่วง min–max	ค่าคาดหวัง 1/(N+1)	mod N ย้าย (คน)	mod N อัตรา
5	1,347	0.135	0.117–0.200	0.167	8,365	0.837
10	910	0.091	0.072–0.101	0.091	9,063	0.906
20	456	0.046	0.039–0.050	0.048	9,551	0.955

ตารางที่ 7.4 อัตราการย้ายเมื่อเพิ่มอาคาร 1 หลัง (ค่ามัธยฐาน)

N	ring เพิ่ม	mod N เพิ่ม	ring ลบ	mod N ลบ
5	13.08	17.58	13.12	17.21
10	11.79	18.07	11.55	18.35
20	11.60	18.91	11.22	18.87

ตารางที่ 7.5 เวลาของการเพิ่มและลบอาคาร (ms, K = 10,000, ค่ามัธยฐาน)



รูปที่ 7.2 อัตราการย้ายเมื่อเพิ่มหรือลบอาคาร 1 หลัง (K = 10,000, V = 32, 3 รอบ) เส้นประคือค่าคาดหวังของ Consistent Hashing

การอภิปรายผล อัตราการย้ายของ Consistent Hashing ใกล้เคียงค่าคาดหวัง 1/(N+1) เมื่อเพิ่มอาคาร และ 1/N เมื่อลบอาคาร และลดลงเมื่อ N มากขึ้น เช่น ที่ N = 20 การเพิ่มอาคารย้ายแขกเพียง 456 คนจาก 10,000 คน (ค่าคาดหวัง 476 คน) ทุกรอบผ่านการตรวจจกติกาว่าแขกที่ย้ายต้องเกี่ยวข้องกับอาคารที่เพิ่มหรือถูกลบเท่านั้น ในทางตรงกันข้าม hash mod N ย้ายแขกถึง 80–96% และอัตรายิ่งสูงขึ้นเมื่อ N มาก ซึ่งตรงกับการวิเคราะห์ในหัวข้อ 6.8 ว่า เมื่อเปลี่ยนจาก N เป็น N+1 มีเพียงสัดส่วน 1/(N+1) ของค่าแฮชที่ยังได้อาคารเดิม ที่ N = 20 hash mod N ย้ายแขกมากกว่า Consistent Hashing ประมาณ 21 เท่า (9,551 เทียบกับ 456 คน)

ค่าของ Consistent Hashing แกว่งระหว่าง salt พอสมควร โดยเฉพาะที่ N = 5 (อัตราการเพิ่มอาคารอยู่ที่ 0.117–0.200) เพราะตำแหน่งจุดบนวงแหวนขึ้นกับ salt และแต่ละอาคารมี virtual node เพียง 32 จุด แต่ค่ามัธยฐานยังใกล้ค่าคาดหวัง ส่วน hash mod N แทบไม่แกว่ง เพราะอัตราการย้ายถูกกำหนดด้วยเลขคณิตมากกว่าการสุ่ม

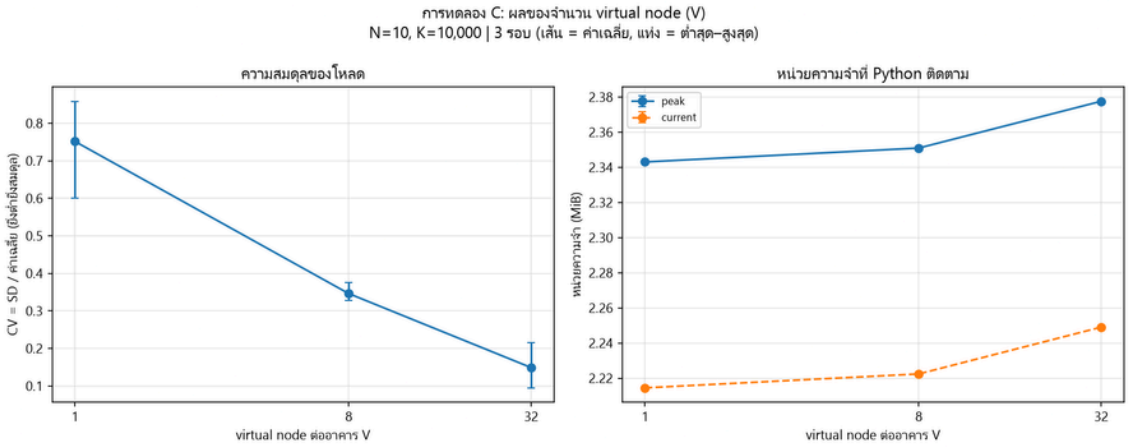
ด้านเวลา Consistent Hashing ใช้ 11.2–13.1 ms ซึ่งน้อยกว่า hash mod N (17.2–18.9 ms) ประมาณ 1.3–1.7 เท่า และไม่เพิ่มตาม N เวลาของ Consistent Hashing ที่ K = 10,000 ใกล้เคียงกับผลคูณของ K กับเวลาคำนวณอาคารจากวงแหวน (10,000 × 1.15 µs ≈ 11.5 ms) และเวลาลบไม่ลดลงตาม N แม้จำนวนแขกของอาคารที่ลบจะเหลือเพียง K/N ซึ่งสนับสนุนข้อสมมติในหัวข้อ 6.10 ข้อ 9 ว่าโปรแกรมตรวจแขกทุกคนทั้งตอนเพิ่มและตอนลบ ซึ่งตรงกับโค้ดของเมธอด __find_moves ที่วนตรวจแขกทั้ง K คนทุกครั้ง

7.4 ผลการทดลองชุด C: ผลของจำนวน virtual node (V)

ชุด C ใช้ Consistent Hashing ที่ N = 10 และ K = 10,000 จึงมีโหลดเฉลี่ยต่ออาคาร 1,000 คน แล้วเปลี่ยน V เป็น 1, 8 และ 32 วัดความสมดุลของโหลดด้วยสัมประสิทธิ์การแปรผัน CV = SD / ค่าเฉลี่ย (ยิ่งต่ำยิ่งสมดุล)

ตัวชี้วัด	V = 1	V = 8	V = 32
โหลดสูงสุดต่ออาคาร (คน)	2,284	1,555	1,286
โหลดต่ำสุดต่ออาคาร (คน)	50	500	774
SD ของโหลด (คน)	795.1	334.5	136.3
CV	0.795	0.335	0.136
ช่วง CV (min–max)	0.601–0.859	0.328–0.376	0.095–0.215
หน่วยความจำ current (MiB)	2.215	2.222	2.249
หน่วยความจำ peak (MiB)	2.343	2.351	2.378
อัตราย้ายเมื่อเพิ่มอาคาร (คาดหวัง 0.091)	0.080	0.046	0.091
อัตราย้ายเมื่อลบอาคาร (คาดหวัง 0.100)	0.148	0.084	0.086

ตารางที่ 7.6 ผลการทดลองชุด C (ค่ามัธยฐานของ 3 รอบ)



รูปที่ 7.3 ผลของจำนวน virtual node (N = 10, K = 10,000, 3 รอบ) กราฟซ้ายแสดงความสมดุลของโหลด (CV) กราฟขวาแสดงหน่วยความจำ current และ peak

การอภิปรายผล CV ลดลงอย่างชัดเจนเมื่อ V เพิ่มขึ้น จาก 0.795 (V = 1) เป็น 0.335 (V = 8) และ 0.136 (V = 32) ที่ V = 1 มีอาคารหนึ่งรับแขกเพียง 50 คน ขณะที่อาคารอื่นรับถึง 2,284 คน แต่ที่ V = 32 ทุกอาคารรับแขก

อยู่ระหว่าง 774 ถึง 1,286 คน เมื่อ V เพิ่มขึ้น 32 เท่า CV ลดลงประมาณ 5.8 เท่า ใกล้เคียงแนวโน้ม $1/\sqrt{V}$ ($\sqrt{32} \approx 5.7$) ผลระหว่าง salt ที่ต่างกันก็แคบลงตาม V (ช่วง CV ที่ V = 1 คือ 0.601–0.859 และที่ V = 32 คือ 0.095–0.215) แสดงว่า virtual node ช่วยให้ผลไม่ขึ้นกับการสุ่มตำแหน่งบนวงแหวน

ในด้านหน่วยความจำ เมื่อ V เพิ่มจาก 1 เป็น 32 ค่า current เพิ่มเพียง 0.035 MiB (ประมาณ 1.6%) เพราะ virtual node เพิ่มจุดบนวงแหวนทั้งหมดเพียง 310 จุด (ประมาณ 117 ไบต์ต่อจุด) การเพิ่ม V จึงให้ความสมดุลที่ดีขึ้นมากโดยเสียพื้นที่เพียงเล็กน้อย ส่วนอัตราการย้ายที่ V = 32 ตรงกับชุด B ที่ N = 10 ทุกค่า เพราะใช้พารามิเตอร์เดียวกัน ที่ V ต่ำ อัตราการย้ายแกว่งมาก เพราะอาคารที่ถูกลบอาจมีแขกมากหรือน้อยกว่าปกติ (เช่น การเพิ่มอาคารที่ V = 1 มีช่วง 0.017–0.129)

กราฟแสดงค่าเฉลี่ยพร้อมช่วงต่ำสุดถึงสูงสุด แต่ตารางรายงานมัธยฐาน ค่าทั้งสองจึงต่างกันเล็กน้อยเมื่อรอบใดรอบหนึ่งผิดปกติ เช่น CV ที่ V = 1 ในกราฟคือ 0.751 (ค่าเฉลี่ย) ส่วนในตารางคือ 0.795 (มัธยฐาน)

7.5 สรุปผลการทดลอง

ผลการทดลองยืนยันการวิเคราะห์ในหัวข้อ 6 ใน 3 ประเด็น ประเด็นแรก ต้นทุนของการจัดแขกและเรียงรายการห้องโตตาม K ในรูปแบบเส้นตรงและ K log K ตามลำดับ ขณะที่การสร้างวงแหวนและการคำนวณอาคารแทบไม่ขึ้นกับ K ประเด็นที่สอง Consistent Hashing ย้ายแขกเพียง K/(N+1) ถึง K/N คนเมื่อเพิ่มหรือลบอาคาร ในขณะที่ hash mod N ย้ายเกือบทั้งหมด ประเด็นที่สาม การเพิ่ม virtual node ช่วยให้โหลดสมดุลขึ้นอย่างมาก (CV ลดจาก 0.795 เป็น 0.136) โดยใช้หน่วยความจำเพิ่มเพียงเล็กน้อย

ตารางที่ 7.7 สรุปการเปรียบเทียบ hash mod N กับ Consistent Hashing โดยแยกค่าที่ได้จากการวิเคราะห์ในบทที่ 6 ออกจากค่าที่วัดได้จริงในการทดลอง ช่องที่เขียนว่า "ไม่ได้วัด" คือหัวข้อที่การทดลองวัดเฉพาะ Consistent Hashing

หัวข้อ	hash mod N: วิเคราะห์	Consistent Hashing: วิเคราะห์	hash mod N: วัดได้จริง	Consistent Hashing: วัดได้จริง
การคำนวณอาคาร	O(1)	O(log NV)	ไม่ได้วัด	1.08–1.20 µs ต่อครั้ง (ชุด A)
การเพิ่ม/ลบอาคาร (เวลา)	O(K log K)	เพิ่ม O(NV log NV + K log NV + B log B), ลบ O(NV + K log NV + B log B)	17.2–18.9 ms (ชุด B)	11.2–13.1 ms (ชุด B)
อัตราการย้าย (ค่าคาดหวัง)	เพิ่ม ≈ N/(N+1), ลบ ≈ 1 − 1/N	เพิ่ม ≈ 1/(N+1), ลบ ≈ 1/N	0.80–0.96 (ชุด B)	0.045–0.202 (ชุด B)
พื้นที่จัดเส้นทาง	O(N) รายการรหัสอาคาร	O(NV) จุดบนวงแหวน	ไม่ได้วัด	เพิ่ม V จาก 1 เป็น 32 (+310 จุด) ใช้หน่วยความจำเพิ่ม 0.035 MiB (ชุด C)
พื้นที่ข้อมูลรวม	O(N + K)	O(NV + K)	ไม่ได้วัด	2.25 MiB ที่ K = 10,000 และ 26.85 MiB ที่ K = 100,000 (ชุด A)
ความสมดุลของโหลด	แขกได้อาคารแบบสุ่มเท่ากัน CV คาดหวัง ≈ √((N−1)/K) ≈ 0.03 ที่ N = 10, K = 10,000	ขึ้นกับความยาวช่วงบนวงแหวน CV ลดลงเมื่อ V เพิ่ม	ไม่ได้วัด	CV 0.795 (V = 1), 0.335 (V = 8), 0.136 (V = 32) (ชุด C)

ตารางที่ 7.7 การเปรียบเทียบ hash mod N กับ Consistent Hashing ระหว่างค่าจากการวิเคราะห์และค่าที่วัดได้จริง

ทั้งสองวิธีมีต้นทุนเพิ่ม/ลบอาคารระดับใกล้เคียงกัน เพราะโปรแกรมตรวจแขกทุกคนเหมือนกัน แต่จำนวนแขกที่ย้ายต่างกันมาก hash mod N ย้ายแขก 80–96% ขณะที่ Consistent Hashing ย้ายเพียง 4.5–20% งานที่ทำต่อแขกที่ย้ายจึงทำให้ Consistent Hashing ใช้เวลาน้อยกว่าจริง แม้ Big-O จะสูงกว่า ในทางกลับกัน hash mod N ใช้พื้นที่จัดเส้นทางน้อยกว่า (N เทียบกับ NV) และตามทฤษฎีกระจายโหลดได้สมดุลกว่า Consistent Hashing จึงแลกพื้นที่ O(NV) และความสมดุลที่ลดลงเล็กน้อยกับการย้ายแขกที่น้อยกว่าหลายสิบเท่าเมื่อจำนวนอาคารเปลี่ยน

ข้อจำกัดของการทดลองคือทำเพียง 3 รอบต่อค่าพารามิเตอร์ ใช้เครื่องเดียว และวัดหน่วยความจำเฉพาะส่วนที่ Python ติดตาม ค่าเวลาจึงเป็นค่าเปรียบเทียบเชิงสัมพัทธ์ ไม่ใช่ค่าสัมบูรณ์ที่ใช้ได้กับเครื่องอื่น

บทที่ 8
การทดสอบความถูกต้อง

8.1 วิธีการทดสอบ

ความถูกต้องของโปรแกรมตรวจด้วยสคริปต์ทดสอบอัตโนมัติที่เขียนด้วยโมดูล unittest ของ Python เก็บไว้ในโฟลเดอร์ tests/ มีทั้งหมด 9 ไฟล์ 147 กรณีทดสอบ สมาชิกแต่ละคนเขียนเทสต์ของส่วนที่ตนพัฒนา สคริปต์รันซ้ำได้ทุกเมื่อด้วยคำสั่งเดียว และใช้เวลาประมาณ 1–2 วินาที

python -m unittest discover -s tests -v

แต่ละกรณีทดสอบสร้างข้อมูลของตัวเองใหม่ทุกครั้ง จึงไม่ขึ้นกับลำดับการรัน การทดสอบที่ต้องควบคุมตำแหน่งบนวงแหวน เช่น ตำแหน่งที่ชนกัน หรือแขกที่อยู่หลังจุดสุดท้าย ใช้ความสามารถของ HashRing ที่รับฟังก์ชันแฮชเป็นพารามิเตอร์ (hash_fn) จึงแทนฟังก์ชันแฮชจริงด้วยฟังก์ชันที่คืนตำแหน่งที่กำหนดไว้ล่วงหน้าได้ ส่วนการทดสอบอื่นใช้ SHA-256 จริง

ไฟล์ทดสอบ	สิ่งที่ตรวจ	จำนวนกรณี
test_hashing.py	รูปแบบข้อความที่นำไปแฮช, ค่าแฮชคงที่ทุกครั้งที่รัน, ตรงกับ 8 ไบต์แรกของ SHA-256, อยู่ในช่วง 64 บิต, ผลของ salt	9
test_guest.py	สูตรเลขห้องตรงกับตารางตัวอย่าง, เลขห้องไม่ซ้ำกัน, ผลเป็นจำนวนเต็มเสมอ, ปฏิเสธรหัสที่ไม่ถูกต้อง, Guest แก้ค่าไม่ได้	8
test_hash_ring.py	วงแหวนขนาดเล็ก, การค้นหาตรงจุด ระหว่างจุด และหลังจุดสุดท้าย (วนกลับต้นวงแหวน), จุดชนกัน, วงแหวนที่มี V > 1, กรณีขอบของวงแหวน	27
test_hotel_system.py	เพิ่มและนำแขกออกทั้งรายคนและเป็นกลุ่ม, ที่อยู่ของแขกเดิมไม่เปลี่ยน, ค้นหาสองทาง, รายการห้องเรียงถูกต้อง, สถิติโหลด, ความสอดคล้องของข้อมูล	19

ไฟล์ทดสอบ	สิ่งที่ตรวจ	จำนวนกรณี
test_migration.py	การย้ายเมื่อเพิ่มหรือลบอาคาร (วงแหวนเล็กและ V = 32), ผลค้นหาหลังย้าย, hash mod N เทียบด้วยรหัสอาคารจริง, รายงานการย้าย	21
test_edge_cases.py	กรณีขอบของอาคารและของแขก, ระบบที่ไม่มีแขก (K = 0)	25
test_csv_export.py	จำนวนแถวใน CSV เท่ากับจำนวนแขกและจำนวนผู้ย้าย, ภาษาไทยใน UTF-8, เขียนไฟล์ไม่ได้แล้วโปรแกรมไม่หยุด	11
test_cli_smoke.py	จำลองการพิมพ์ในเมนูทุกข้อ, พิมพ์ผิดและกรณีขอบไม่ทำให้โปรแกรมหยุดทำงาน	14
test_experiment_csv.py	ไฟล์ผลการทดลองครบทุกพารามิเตอร์และทุก salt, อัตราย้ายเท่ากับจำนวนย้ายหาร K, ผลสรุปตรงกับผลดิบ	13
รวม		147

ตารางที่ 8.1 ไฟล์ทดสอบและสิ่งที่ตรวจ

8.2 การตรวจตรรกะด้วยวงแหวนขนาดเล็ก

ก่อนทดสอบกับค่าแฮชจริง กลุ่มตรวจตรรกะการเลือกอาคารและการย้ายด้วยวงแหวนขนาดเล็กที่คำนวณคำตอบด้วยมือได้ วงแหวนนี้มีขนาด 100 ตำแหน่ง และ V = 1 วางอาคาร A ที่ตำแหน่ง 20, B ที่ 50 และ C ที่ 80 แล้ววางแขก 7 คน คือ (1, 1) ถึง (1, 7) ไว้ที่ตำแหน่ง 10, 20, 25, 35, 50, 70 และ 90 ตามลำดับ ตำแหน่งเหล่านี้กำหนดผ่านฟังก์ชันแฮชจำลอง ไม่ได้มาจาก SHA-256 เพราะจุดประสงค์คือตรวจตรรกะ ในโค้ดใช้รหัสอาคาร A, B, C, D เป็น 1, 2, 3, 4 กติกาการเลือกอาคารคือ ให้แขกพักที่จุดแรกที่มีตำแหน่งมากกว่าหรือเท่ากับตำแหน่งของแขก ถ้าไม่มีจุดใดเลยให้วนกลับไปจุดแรกของวงแหวน กรณีเพิ่ม D และกรณีลบ B ต่างเริ่มจากสถานะ A, B, C ใหม่

แขก	ตำแหน่ง	ห้อง	เริ่มต้น (A,B,C)	เพิ่ม D ที่ 35	ลบ B
(1, 1)	10	1	A	A	A
(1, 2)	20	3	A	A	A
(1, 3)	25	6	B	D	C
(1, 4)	35	10	B	D	C
(1, 5)	50	15	B	B	C
(1, 6)	70	21	C	C	C
(1, 7)	90	28	A	A	A

ตารางที่ 8.2 อาคารของแขกแต่ละคนบนวงแหวนขนาดเล็ก (ตัวหนา และขีดเส้นใต้ = แขกที่ย้าย)

สถานะเริ่มต้นครอบคลุมกรณีสำคัญของการค้นหา ได้แก่ แขก (1, 2) ที่ตำแหน่ง 20 และแขก (1, 5) ที่ตำแหน่ง 50 อยู่ตรงจุดของอาคารพอดี จึงพักที่อาคาร A และ B ตามลำดับ ส่วนแขก (1, 7) ที่ตำแหน่ง 90 อยู่หลังจุดสุดท้าย (C ที่ 80) จึงวนกลับไปพักที่อาคาร A

เมื่อเพิ่มอาคาร D ที่ตำแหน่ง 35 มีแขกย้าย 2 คน คือ (1, 3) และ (1, 4) ทั้งคู่ย้ายจาก B เข้า D ซึ่งเป็นอาคารใหม่ อัตราการย้ายเท่ากับ 2/7 ≈ 0.2857 เมื่อลบอาคาร B มีแขกย้าย 3 คน คือ (1, 3), (1, 4) และ (1, 5) ทั้งหมดเป็นแขกของ B และย้ายไปที่ C ซึ่งเป็นอาคารถัดไปตามเข็มนาฬิกา แขกในอาคารอื่นไม่ย้ายเลย ทั้งสองกรณี หมายเลขห้องของแขกที่ย้ายยังเป็นเลขเดิม และ check_integrity คืนค่า True

ผลทั้งหมดในตารางที่ 8.2 ตรวจด้วยกรณีทดสอบ 12 กรณี ได้แก่ TestSmallRingExample (3 กรณี) และ TestLocate (5 กรณี) ใน test_hash_ring.py ซึ่งตรวจระดับวงแหวน และ SmallRingExample91 (4 กรณี) ใน test_migration.py ซึ่งตรวจผ่าน HotelSystem ทั้งจำนวนผู้ย้าย รายชื่อผู้ย้าย อาคารเดิม อาคารใหม่ หมายเลขห้อง และอัตราการย้าย ทุกกรณีผ่านดังรูปที่ 8.1

```
PS C:\Users\Msi\Desktop\second year\OOD Project> python -m unittest tests.test_hash_ring.TestSmallRingExample tests.test_hash_ring.TestLocate test
s.test_migration.SmallRingExample91 -v
test_add_d_moves_two_guests_into_d_only (tests.test_hash_ring.TestSmallRingExample.test_add_d_moves_two_guests_into_d_only) ... ok
test_initial_assignment (tests.test_hash_ring.TestSmallRingExample.test_initial_assignment) ... ok
test_remove_b_moves_three_guests_only_from_b (tests.test_hash_ring.TestSmallRingExample.test_remove_b_moves_three_guests_only_from_b) ... ok
test_after_last_point_wraps_around (tests.test_hash_ring.TestLocate.test_after_last_point_wraps_around) ... ok
test_before_first_point (tests.test_hash_ring.TestLocate.test_before_first_point) ... ok
test_between_points (tests.test_hash_ring.TestLocate.test_between_points) ... ok
test_empty_ring_rejected (tests.test_hash_ring.TestLocate.test_empty_ring_rejected) ... ok
test_exactly_on_point (tests.test_hash_ring.TestLocate.test_exactly_on_point) ... ok
test_add_D_moves_two_into_D_only (tests.test_migration.SmallRingExample91.test_add_D_moves_two_into_D_only) ... ok
test_initial_assignment (tests.test_migration.SmallRingExample91.test_initial_assignment) ... ok
test_rate_matches_moved_over_K (tests.test_migration.SmallRingExample91.test_rate_matches_moved_over_K) ... ok
test_remove_B_moves_three_all_from_B (tests.test_migration.SmallRingExample91.test_remove_B_moves_three_all_from_B) ... ok

----------------------------------------------------------------------
Ran 12 tests in 0.001s

OK
```

รูปที่ 8.1 ผลการรันกรณีทดสอบของวงแหวนขนาดเล็ก

8.3 รายการทดสอบหลัก

หัวข้อนี้สรุปว่ากติกาหลักแต่ละข้อของระบบตรวจด้วยกรณีทดสอบใด การทดสอบที่ใช้ค่าแฮชจริงส่วนใหญ่สร้างระบบ 5 อาคาร V = 32 และแขก 10 ช่องทาง ช่องทางละ 300 คน (K = 3,000) ใช้ salt “0” ส่วนการทดสอบที่ต้องควบคุมตำแหน่งบนวงแหวนใช้ฟังก์ชันแฮชจำลองตามที่อธิบายในหัวข้อ 8.1

กติกาที่ตรวจ	วิธีตรวจ	กรณีทดสอบ
แขกใหม่ไม่ทำให้แขกเดิมย้าย	บันทึกที่อยู่ของแขกทุกคน แล้วเพิ่มแขก 50 คนในช่องทางเดิม (ช่องทาง 1 ลำดับ 301–350) และอีก 50 คนในช่องทางใหม่ (ช่องทาง 11) จากนั้นตรวจว่าที่อยู่ของแขกเดิมไม่เปลี่ยนแม้แต่คนเดียว	test_new_guests_do_not_move_existing_ones, test_existing_guest_address_unchanged_when_new_guest_added
เลขห้องถูกต้องและไม่ซ้ำ	เทียบกับตัวอย่าง (1,1)→1, (2,1)→2, (1,2)→3, (3,1)→4, (2,2)→5 · ไล่รหัสแขกทั้ง 576 รหัส (c, s = 1–24) ว่าไม่มีเลขห้องซ้ำ · ตรวจว่าผลเป็นจำนวนเต็มเสมอ	test_table_values_from_readme, test_no_collisions_over_a_grid, test_result_is_pure_integer, test_different_codes_never_share_room
ปฏิเสธรหัสแขกซ้ำ	เพิ่มรหัสเดิมซ้ำ และเพิ่มเป็นกลุ่มที่ทับรหัสเดิมเพียงบางรายการ (รวมกรณีทับเฉพาะรายการสุดท้าย) แล้วตรวจว่าจำนวนแขกและข้อมูลเดิมไม่เปลี่ยน	test_add_guest_duplicate_rejected, test_add_guests_group_partial_duplicate_rejects_whole_group, test_duplicate_guest_not_overwritten, test_group_with_partial_overlap_rejected_whole, test_group_overlap_only_last_item
ค้นหาบนวงแหวนถูกทุกตำแหน่ง	ค้นตรงจุดของอาคารพอดี, ระหว่างจุด, ก่อนจุดแรก, หลังจุดสุดท้าย (ต้องวนกลับต้นวงแหวน) และวงแหวนว่าง	TestLocate (5 กรณี)

กติกาที่ตรวจ	วิธีตรวจ	กรณีทดสอบ
จุดบนวงแหวนชนกัน	สองอาคารอยู่ตำแหน่งเดียวกัน ต้องเก็บไว้ครบทั้งสองจุดและเลือกอาคารที่ node_id น้อยกว่า · ถ้าจุดของอาคารเดียวกันชนกัน ใช้ j ที่น้อยกว่า · ลบอาคารหนึ่งแล้วอีกอาคารยังอยู่ · แขกหลายคนที่ตำแหน่งเดียวกันได้ห้องต่างกัน	TestCollisions (4 กรณี)
เพิ่มและลบอาคารเมื่อ V > 1	ใช้ SHA-256 จริง ตรวจว่าเมื่อเพิ่มอาคาร ผู้ย้ายเข้าอาคารใหม่เท่านั้น และเมื่อลบอาคาร ย้ายเฉพาะแขกของอาคารนั้น · จำนวนแขกรวมไม่หายไม่เพิ่ม · จำนวนที่รายงานเท่ากับจำนวนรายการย้าย · เลขห้องคงเดิม · ค้นหาสองทางยังถูกหลังย้าย · เพิ่มแล้วลบอาคารเดิมได้สถานะกลับมาเหมือนเดิมทุกคน	RingMigrationV32 (7 กรณี), TestRealHashWithVirtualNodes (7 กรณี)
ค่าแฮชคงที่ทุกครั้งที่เปิดโปรแกรม	เทียบกับค่าที่จดไว้ล่วงหน้า เช่น hash64("guest:1:1") = 3062031282815716176 และเทียบกับ 8 ไบต์แรกของ SHA-256 ที่คำนวณอีกวิธี	test_fixed_values_across_runs, test_matches_sha256_first_8_bytes
การเปรียบเทียบกับ hash mod N	นับการย้ายด้วยรหัสอาคารจริง ไม่ใช่ดัชนีในรายการ · ใช้ salt และค่าแฮชชุดเดียวกับวงแหวน · hash mod N ย้ายแขกมากกว่า Consistent Hashing	ModNMigration (5 กรณี)

ตารางที่ 8.3 กติกาหลักและกรณีทดสอบที่ใช้ตรวจ

กรณีทดสอบ test_fixed_values_across_runs ป้องกันการใช้ hash() ของ Python เป็นแฮชหลัก ถ้าใช้ hash() ค่าจะถูกสุ่มใหม่ทุกครั้งที่เปิดโปรแกรมเทสต์นี้จะไม่ผ่านส่วนtest_add_then_remove_same_node_restores_everything เพิ่มอาคาร 6 แล้วลบอาคาร 6 ทันที แล้วตรวจว่าแขกทั้ง 3,000 คนกลับมาอยู่ที่อาคารและห้องเดิมทุกคน กรณีนี้ยืนยันว่าการเพิ่มอาคารแตะเฉพาะจุดของอาคารใหม่ และการลบอาคารลบเฉพาะจุดของอาคารนั้น จุดของอาคารอื่นไม่เปลี่ยน

8.4 การทดสอบกรณีขอบ

กรณีขอบคือข้อมูลนำเข้าที่อยู่ตรงเส้นแบ่งของความถูกต้อง เช่น ค่าที่ซ้ำ ค่าที่ไม่มีอยู่จริง เหลือเพียงชิ้นเดียว หรือไม่มีข้อมูลเลย โปรแกรมที่เขียนตรงไปตรงมามักผิดในกรณีเหล่านี้ ทุกกรณีต้องแจ้งเหตุผล คงข้อมูลเดิมไว้ และโปรแกรมต้องไม่หยุดทำงาน การทดสอบส่วนใหญ่ในไฟล์ test_edge_cases.py เริ่มจากระบบ 4 อาคาร V = 8 และแขก 3 ช่องทาง ช่องทางละ 30 คน (K = 90) ใช้ salt “0” แต่ละกรณีบันทึกสถานะทั้งหมดของระบบก่อนสั่งคำสั่ง ได้แก่ ที่อยู่ของแขกทุกคน รายการห้อง จำนวนแขกต่ออาคาร รายชื่ออาคาร และจำนวนจุดบนวงแหวน เมื่อคำสั่งถูกปฏิเสธ จะเทียบสถานะหลังคำสั่งกับก่อนคำสั่งว่าเหมือนเดิมทุกส่วน และ check_integrity คืนค่า True

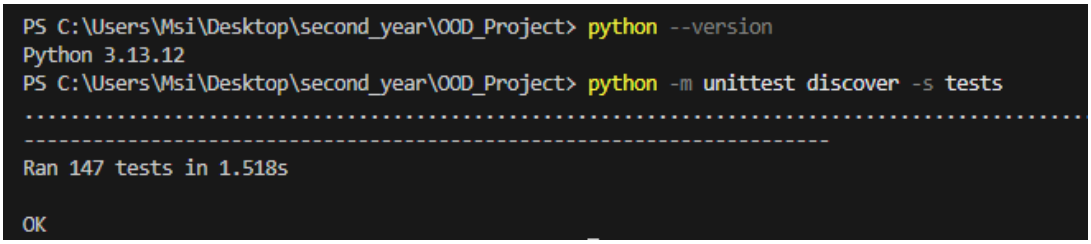
กรณีขอบ	ผลที่ตรวจ	กรณีทดสอบ
เพิ่มรหัสอาคารที่มีอยู่แล้ว	ปฏิเสธ จำนวนจุดบนวงแหวนยังเท่าเดิม (ไม่ใส่จุดซ้ำ) และเมื่อลบอาคารนั้นภายหลัง จุดถูกลบออกหมด ไม่มีจุดตกค้าง	test_add_existing_node, test_duplicate_node_does_not_double_ring_points, test_add_existing_node_rejected_and_ring_unchanged
ลบรหัสอาคารที่ไม่มีอยู่	ปฏิเสธ ข้อมูลไม่เปลี่ยน และไม่มีรายงานจำนวนคนย้ายออกมา	test_remove_missing_node, test_remove_missing_node_reports_nothing, test_remove_missing_node_rejected_and_ring_unchanged
ลบอาคารสุดท้าย	ปฏิเสธทั้งกรณีที่อาคารสุดท้ายมีแขก (ลบจนเหลืออาคารเดียวที่มีแขกครบ 90 คน) และกรณีไม่มีแขก หลังถูกปฏิเสธยังค้นหาและเพิ่มแขกได้ตามปกติ	test_remove_last_node_with_guests, test_remove_last_node_without_guests, test_remove_last_node_rejected
เพิ่มรหัสแขกที่ซ้ำ	ปฏิเสธ ไม่เขียนทับข้อมูลแขกเดิม	test_duplicate_guest_not_overwritten

กรณีขอบ	ผลที่ตรวจ	กรณีทดสอบ
นำแขกที่ไม่มีอยู่ออก	ปฏิเสธ จำนวนแขก K ไม่ลดลง	test_remove_missing_guest_keeps_K, test_remove_nonexistent_guest_raises_and_k_unchanged
ค้นหาแล้วไม่พบ	คืนค่า None ทั้งการค้นด้วยรหัสแขกและด้วยที่อยู่ห้อง และเมนูแสดงคำว่า "ไม่พบ"	test_find_missing_returns_none, test_find_returns_none_when_not_found, test_find_with_invalid_code_finds_nothing
ไม่มีแขกในระบบ (K = 0)	สถิติแสดงทุกอาคารเป็น 0 คน ค่าสูงสุด ต่ำสุด เฉลี่ย และ SD เป็น 0 โดยไม่คำนวณ CV · เพิ่มหรือลบอาคารแล้วรายงานย้าย 0 คน โดยไม่คำนวณอัตรา · นำแขกออกจนหมดแล้วยังทำงานต่อได้	test_K_zero_stats, test_load_stats_when_no_guests, test_empty_system_reports_zero_and_no_rate, test_all_guests_removed_then_stats_and_migration
ค่าที่ไม่ถูกต้อง	ปฏิเสธรหัสอาคาร 0, −1, 1.5, “2”, None, True · ปฏิเสธจำนวนแขกติดลบ · เพิ่มแขกเป็นกลุ่มจำนวน 0 คน ข้อมูลไม่เปลี่ยน	test_invalid_node_ids, test_invalid_inputs, test_add_guests_negative_count_is_invalid, test_group_count_zero_changes_nothing, test_create_validation

ตารางที่ 8.4 กรณีขอบและผลที่ตรวจ

8.5 ผลการทดสอบ

รันสคริปต์ทดสอบทั้งชุดบนเครื่องที่ใช้พัฒนา (Windows 11, Python 3.13.12) ด้วยคำสั่ง python -m unittest discover -s tests จากโฟลเดอร์หลักของโปรเจกต์ ผลคือผ่านทั้งหมด 147 กรณีจาก 147 กรณี ไม่มีกรณีที่ไม่ผ่าน (failure) หรือเกิดข้อผิดพลาดระหว่างรัน (error) ใช้เวลาประมาณ 1–2 วินาที ดังรูปที่ 8.2



รูปที่ 8.2 ผลการรันสคริปต์ทดสอบอัตโนมัติทั้งชุด

ไฟล์ทดสอบ	จำนวนกรณี	ผ่าน	ไม่ผ่าน
test_hashing.py	9	9	0
test_guest.py	8	8	0
test_hash_ring.py	27	27	0
test_hotel_system.py	19	19	0
test_migration.py	21	21	0
test_edge_cases.py	25	25	0

ไฟล์ทดสอบ	จำนวนกรณี	ผ่าน	ไม่ผ่าน
test_csv_export.py	11	11	0
test_cli_smoke.py	14	14	0
test_experiment_csv.py	13	13	0
รวม	147	147	0

ตารางที่ 8.5 สรุปผลการทดสอบแยกตามไฟล์

ผลการรันแบบละเอียด ซึ่งแสดงชื่อและผลของกรณีทดสอบทีละกรณี บันทึกไว้ในไฟล์ docs/test_results.txt ที่ส่งมาพร้อมโปรแกรม ผู้ตรวจรันซ้ำเพื่อดูผลเดียวกันได้ด้วยคำสั่ง python -m unittest discover -s tests -v เนื่องจากทุกกรณีผ่าน จึงไม่มีกรณีที่ต้องอธิบายเพิ่มเติม สคริปต์ทดสอบตรวจความถูกต้องของผลลัพธ์เท่านั้น ไม่ได้ตรวจความเร็ว ส่วนเวลาและหน่วยความจำวัดแยกในบทที่ 7 กรณีทดสอบใน test_experiment_csv.py ตรวจเพียงว่าไฟล์ผลการทดลองมีข้อมูลครบและสอดคล้องกัน เช่น อัตราการย้ายเท่ากับจำนวนที่ย้ายหารด้วย K และผลสรุปตรงกับผลดิบของทั้ง 3 รอบ

บทที่ 9
ข้อจำกัดและแนวทางพัฒนา

9.1 ข้อจำกัดของงานนี้

1) ขอบเขตการทำงาน

โปรแกรมรันเป็นโปรเซสเดียวบนเครื่องเดียว อาคารแต่ละหลังเป็นเพียงรหัสในพจนานุกรมของ HotelSystem ไม่มีเครือข่าย ไม่จำลองเครื่องล่ม และไม่มีการกู้ข้อมูล ตามขอบเขตในหัวข้อ 1.3 ข้อมูลทั้งหมดอยู่ในหน่วยความจำ เมื่อปิดโปรแกรมข้อมูลแขกจะหายไป ยกเว้นที่ส่งออกเป็นไฟล์ CSV โปรแกรมมีเฉพาะการส่งออก ไม่มีการอ่านข้อมูลกลับจากไฟล์ ส่วนติดต่อผู้ใช้เป็นเมนูข้อความ ไม่มี GUI และไม่มีภาพวงแหวน

2) การหาแขกที่ต้องย้ายใช้แบบ ก (ตรวจทุกคน)

เมธอด __find_moves วนดูแขกทั้ง K คนและเรียก node_for ให้ทุกคน จึงมีต้นทุน O(K log M) ทุกครั้งที่เพิ่มหรือลบอาคาร แม้ผู้ย้ายจริงมีเพียงส่วนน้อย ตัวอย่างจากการทดลอง B ที่ N = 10, K = 10,000 อัตราย้ายมัธยฐานอยู่ที่ 0.091 หรือราว 910 คน แต่โปรแกรมต้องตรวจแขก 10,000 คน

3) การปรับวงแหวนทำใหม่ทั้งรายการ

add_node เพิ่ม V จุดต่อท้ายแล้วเรียง list ทั้งหมดใหม่ และสร้าง list ตำแหน่งใหม่อีกครั้ง รวม O(M log M) ในกรณีเลวร้ายที่สุด (Timsort ทำงานจริงใกล้ O(M + V log V) เพราะ list เดิมเรียงอยู่แล้ว) ส่วน remove_node กรอง list ทั้งหมดใน O(M) ซึ่งยอมรับได้เมื่อ M = N × V ไม่ใหญ่มาก แต่จะช้าลงเมื่อจำนวนอาคารและ V เพิ่มขึ้น

4) หน่วยความจำ

ระบบเก็บ list ของจุดบนวงแหวนสองชุด (จุดเต็มและตำแหน่งล้วน) พร้อม dict ตำแหน่งแขกและ dict ดัชนีห้อง พื้นที่หลักจึงเป็น O(NV + K) และมีพื้นที่ชั่วคราว O(จำนวนผู้ย้าย) สำหรับรายการย้ายระหว่างคำสั่ง ค่าที่วัดด้วย tracemalloc เป็นเฉพาะหน่วยความจำที่ Python ติดตาม ไม่ใช่หน่วยความจำของทั้งโปรเซส

5) ความสมดุลของโหลดและอัตราการย้ายเป็นค่าสุ่มตามตำแหน่งแฮช

ที่ N = 10, K = 10,000 ค่า CV มัธยฐานเท่ากับ 0.795 เมื่อ V = 1 ลดเหลือ 0.335 เมื่อ V = 8 และ 0.136 เมื่อ V = 32 แต่ที่ V = 32 ค่า CV ในแต่ละรอบยังกว้าง (0.095–0.215) ส่วนอัตราย้ายก็ไม่เท่าค่าคาดหวังทุกครั้ง เช่น ที่ N = 5 การเพิ่มอาคารมีค่ามัธยฐาน 0.135 ต่ำกว่าค่าคาดหวัง 1/6 ≈ 0.167 (ช่วง 0.117–0.200) เพราะค่าคาดหวังสมมติว่าแฮชกระจายสม่ำเสมอ แต่จุดจริงมีไม่กี่จุดและตำแหน่งเปลี่ยนตาม salt

6) ข้อจำกัดของการทดลอง

ทำชุดละ 3 รอบ (salt 0, 1, 2) บนเครื่องเดียว (Windows 11, Python 3.13) จึงสรุปความแปรปรวนได้เพียงคร่าว ๆ เวลาที่วัดได้ขึ้นกับเครื่อง K สูงสุดที่ทดลองคือ 100,000 และทดลอง V แค่ 1, 8, 32 จึงยังไม่ทราบจุดสมดุลระหว่าง CV กับหน่วยความจำเมื่อ V สูงกว่านี้

7) ข้อจำกัดของการออกแบบ

- ค่า V คงที่ตลอดอายุของวงแหวน ถ้าจะเปลี่ยนต้องสร้างวงแหวนใหม่
- ทุกอาคารมีน้ำหนักเท่ากัน ยังรองรับอาคารที่มีขนาดไม่เท่ากันไม่ได้
- node_id, c, s ต้องเป็นจำนวนเต็มบวก
- เลขห้องจาก Cantor pairing โตตามกำลังสองของ (c + s) รหัสที่ใหญ่มากจะได้เลขห้องใหญ่ตามไปด้วย
- ไม่รองรับการใช้งานพร้อมกันหลาย thread

9.2 แนวทางพัฒนาต่อ

1. เปลี่ยนเป็นแบบ ข (หาเฉพาะช่วงที่ได้รับผลกระทบ) เก็บแขกเรียงตามตำแหน่งแฮชเพิ่มอีกชุด แล้วใช้ bisect ดึงเฉพาะแขกในช่วงของ virtual node ที่เพิ่มหรือลบ ต้นทุนจะลดจาก O(K log M) เหลือประมาณ O(V log K + จำนวนผู้ย้าย) แต่ต้องดูแลดัชนีเพิ่มอีกชุดให้สอดคล้อง และต้องจัดการช่วงที่วนข้ามปลายวงแหวนกับจุดชนกันให้ถูก
2. ปรับวิธีแทรกจุดในวงแหวน ใช้การผสานสอง list ที่เรียงแล้ว (O(M + V log V)) แทนการเรียงใหม่ทั้งรายการ
3. เพิ่ม virtual node ตามน้ำหนักอาคาร ให้อาคารใหญ่รับแขกมากกว่าตามสัดส่วน
4. เพิ่มงานเสริม วาดภาพวงแหวนจาก points() ซึ่งมีอยู่แล้ว และเพิ่มการอ่านข้อมูลแขกกลับจาก CSV
5. ขยายการทดลอง ลอง V = 64, 128, 256 เพิ่มจำนวน salt ให้มากกว่า 3 รอบเพื่อให้ช่วงความแปรปรวนน่าเชื่อถือขึ้น และวัดเวลาเทียบแบบ ก กับแบบ ข หลังเขียนแบบ ข เสร็จ

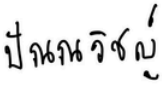
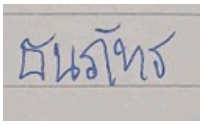
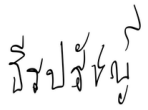
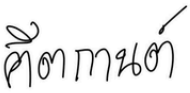
แหล่งอ้างอิง

1. เอกสารข้อกำหนดโครงงานรายวิชา Object Oriented Data Structure ภาคการศึกษาที่ 1/2569 เรื่อง โรงแรมอนันต์แบบกระจาย (Distributed Hilbert's Hotel) ด้วยเทคนิค Consistent Hashing — ใช้เป็นกติกาของระบบ ขอบเขตงาน ชุดการทดลอง และรายการทดสอบความถูกต้อง
2. Hilbert's Hotel: https://youtu.be/HLTjDXT9SqQ — ใช้ทบทวนปัญหาโรงแรมอนันต์แบบดั้งเดิมในหัวข้อ 1.1
3. Consistent Hashing (ByteByteGo): https://www.youtube.com/watch?v=UF9Iqmg94tk — ใช้ศึกษาหลักการวงแหวนแฮชและ virtual node ก่อนออกแบบคลาส HashRing (บทที่ 3 และ 4)
4. Python hashlib: https://docs.python.org/3/library/hashlib.html — ใช้ฟังก์ชัน sha256 ใน hash64 เพื่อให้ตำแหน่งบนวงแหวนคงที่ทุกครั้งที่รันโปรแกรม (หัวข้อ 4.1)
5. Python bisect: https://docs.python.org/3/library/bisect.html — ใช้ bisect_left หาจุดถัดไปบนวงแหวน และใช้ประกอบการวิเคราะห์ต้นทุนการค้นหาและการแทรกใน list (หัวข้อ 3.1, 6.3 และ 6.8)
6. Python tracemalloc: https://docs.python.org/3/library/tracemalloc.html — ใช้วัดหน่วยความจำ current และ peak ในการทดลอง (หัวข้อ 7.1)
7. Python time.perf_counter: https://docs.python.org/3/library/time.html#time.perf_counter — ใช้จับเวลาแต่ละขั้นตอนในการทดลอง (หัวข้อ 7.1)

ภาคผนวก

ลำดับ	รหัสนักศึกษา	ชื่อ-นามสกุล	ส่วนที่พัฒนา	ส่วนที่ทดสอบ	สัดส่วน (ร้อยละ)
1	68010691	นายปัณณวิชญ์ กิติวงศ์ไพศาล	วงแหวนและการเลือกอาคาร (hashing.py, hash_ring.py)	test_hashing.py, test_hash_ring.py	25
2	68010467	นายธนภัทร แคล้วคลาด	ข้อมูลแขกและเลขห้อง (guest.py, hotel_system.py ส่วนแขก)	test_guest.py, test_hotel_system.py	25
3	68010544	นายธีรปรัชญ์ ตั้นพันธุ์	เพิ่ม/ลบอาคารและการย้าย, hash mod N, CSV (hotel_system.py ส่วนอาคาร, mod_n.py, models.py, csv_export.py)	test_migration.py, test_edge_cases.py, test_csv_export.py	25
4	68010125	นายคีตกานต์ แซ่ลิ่ม	การทดลองและส่วนติดต่อผู้ใช้ (experiments/, cli/menu.py, main.py)	test_cli_smoke.py, test_experiment_csv.py	25

ตาราง ก

รหัสนักศึกษา	ชื่อ-นามสกุล	ลายเซ็น
68010691	นายปัณณวิชญ์ กิติวงศ์ไพศาล	
68010467	นายธนภัทร แคล้วคลาด	
68010544	นายธีรปรัชญ์ ตั้นพันธุ์	
68010125	นายศีตกานต์ แซ่ลิ่ม	

ตาราง ข